



SAPIENZA
UNIVERSITÀ DI ROMA

Big Data Computing

Appunti del corso

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Corso di Laurea Magistrale in Computer Science

Docente

Prof. Gabriele Tolomei

Appunti di

Alessio Cavedal

GitHub: <https://github.com/AlessioCavedal>

Anno Accademico 2026/2027

Indice

Il corso	ii
I Infrastruttura per i big data	1
1 Il paradigma scale-out e lo storage distribuito	2
1.1 Perché scalare in orizzontale	2
1.2 Principi di progetto dei file system distribuiti	5
1.3 Google File System	7
1.4 HDFS	11
1.5 Località dei dati e replicazione	15
1.6 Gestione dei guasti	17
1.7 Dallo storage al calcolo	21
2 Il modello di esecuzione MapReduce	23
2.1 L'astrazione MapReduce	23
2.2 Architettura di esecuzione	26
2.3 Shuffle, sort e reduce	28
2.4 Tolleranza ai guasti e straggler	30
2.5 Punti di forza e limiti	33
3 Apache Spark e i grafi di dataflow	36
3.1 Oltre la pipeline a due stadi	36
3.2 Resilient Distributed Dataset	39
3.3 Trasformazioni e azioni	41
3.4 Valutazione pigra e DAG di esecuzione	43
3.5 Dipendenze strette e ampie	45
3.6 Lineage e tolleranza ai guasti	47
3.7 Il modello di esecuzione completo	50

Il corso

Il corso è diviso in quattro parti.

1. **Infrastruttura per i big data** (*Big Data Infrastructure*): storage scale-out, tolleranza ai guasti tramite *lineage*, MapReduce, esecuzione a DAG in memoria (Spark), *streaming*.
2. **Rappresentazioni dei dati ad alta dimensionalità** (*High-Dimensional Data Representations*): geometria degli spazi ad alta dimensione, riduzione della dimensionalità (Johnson–Lindenstrauss), layout colonnari, *embedding* di testi, immagini e grafi.
3. **Problemi classici senza apprendimento** (*Classical Non-Learning Tasks*): indicizzazione su scala web, teoria del campionamento, *Approximate Query Processing*, *sketch* sub-lineari (HyperLogLog, Count-Min), *Locality-Sensitive Hashing*.
4. **Sistemi di apprendimento scale-out** (*Scale-Out Learning Systems*): ERM su larga scala, SGD parallelo e asincrono, *Parameter Server*, *Federated Learning*, ottimizzazione della banda di memoria.

Testi consigliati. Il materiale esce sul sito del corso, e a lezione si citano diversi articoli di ricerca. Libri consigliati, non obbligatori:

- *Mining of Massive Datasets*, Leskovec, Rajaraman e Ullman (gratuito online);
- *Foundations of Data Science*, Blum, Hopcroft e Kannan (gratuito online);
- *Designing Data-Intensive Applications*, Kleppmann;
- *Introduction to Information Retrieval*, Manning, Raghavan e Schütze;
- *Understanding Machine Learning*, Shalev-Shwartz e Ben-David.

Parte I

Infrastruttura per i big data

Capitolo 1

Il paradigma scale-out e lo storage distribuito

Una macchina infinitamente veloce e affidabile non esiste: il calcolo sui big data comincia dall'accettare questo fatto e dal progettare attorno a esso. Questo capitolo costruisce le fondamenta di tutto il modulo, cioè lo storage distribuito su cui poggeranno MapReduce, Spark e lo *streaming*. Si parte dal perché conviene crescere aggiungendo macchine invece di potenziarne una; si passa ai principi di progetto dei file system distribuiti e a due sistemi concreti, GFS e il suo erede open source HDFS; si chiude con tre temi trasversali: località dei dati, replicazione e gestione dei guasti.

1.1 Perché scalare in orizzontale

I sistemi per i big data esistono perché i volumi di dati superano ciò che una singola macchina può memorizzare o elaborare. Qualche esempio:

- i *crawl* del web, cioè le raccolte di pagine scaricate dai motori di ricerca: centinaia di miliardi di pagine, petabyte di HTML grezzo;
- log e telemetria: dati di sensori e *clickstream* (le sequenze di clic degli utenti), che si accumulano al ritmo di terabyte al giorno;
- dati scientifici e aziendali: genomica, astronomia, log delle transazioni delle grandi imprese, fino alla scala degli exabyte.

Se un solo disco, una sola macchina e un solo punto di guasto non bastano a contenere e servire questi dati, quale alternativa strutturale esiste?

1.1.1 I limiti della singola macchina

I dati sono cresciuti molto più in fretta della capacità di qualunque macchina. La capacità dei dischi raddoppia ogni qualche anno, i dati ogni qualche mese; alcuni carichi, come gli indici del web o le reti di sensori, producono dati più in fretta di quanto un singolo disco riesca anche solo a scriverli.

Un server, per quanto potente, ha poi un tetto fisico su ogni risorsa:

- **CPU**: un numero finito di core per socket e di socket per scheda madre;

- **RAM:** un numero finito di slot DIMM e di memoria indirizzabile;
- **disco:** un numero finito di alloggiamenti per i dischi e una banda di I/O finita per controller;
- **rete:** la banda finita delle schede di rete, in entrata e in uscita.

Anche una macchina «grande» resta limitata: sposta il tetto più in alto, ma non lo elimina.

1.1.2 Scalabilità verticale

Definizione (scalabilità verticale). La **scalabilità verticale** (*vertical scaling*, o *scale up*) consiste nel rendere più grande una singola macchina: più core, più RAM, dischi più veloci o più capienti sullo stesso nodo.

Il vantaggio è la semplicità: spesso il software non va toccato, perché la macchina semplicemente fa di più, e concettualmente c'è una scatola più grande al posto di tante piccole. Il problema è fin dove si può arrivare, e a che prezzo.

Il primo limite è il costo, che nella fascia alta cresce molto più che linearmente (Tabella 1.1). L'hardware di pregio (più socket, interconnessioni speciali, memorie specializzate) si paga carissimo, e oltre una certa soglia raddoppiare la capacità costa ben più del doppio.

Tabella 1.1. Capacità e costo relativi delle configurazioni di fascia alta.

Configurazione	Capacità relativa	Costo relativo
Server commodity	1×	1×
Server di fascia alta	4×	oltre 10×
Classe mainframe, top di gamma	16×	oltre 100×

Il secondo limite è fisico: una scheda madre ha un numero massimo di socket e di slot DIMM, e una macchina ha un unico involucro di alimentazione e raffreddamento. A un certo punto nessuna cifra compra una macchina più grande per un dato carico. La scalabilità verticale fa guadagnare tempo, ma non è una strategia di lungo periodo per dati su scala web o exabyte.

1.1.3 Scalabilità orizzontale

Definizione (scalabilità orizzontale). La **scalabilità orizzontale** (*horizontal scaling*, o *scale out*) consiste nell'aggiungere macchine invece di ingrandirle: il carico si distribuisce su molti nodi indipendenti, ciascuno di capacità modesta, e la capacità del *cluster* cresce aggiungendo nodi, non potenziandone uno.

È la strategia alla base di praticamente tutti i sistemi per i big data. L'idea chiave dietro sistemi come GFS e HDFS è costruire i cluster con macchine **commodity**, cioè economiche, standard e inaffidabili, invece che con poche macchine costose e affidabili:

- migliaia di nodi economici possono costare, nel complesso, meno di pochi nodi di pregio;
- il guasto del singolo nodo diventa un evento previsto, non un'eccezione;
- l'affidabilità diventa responsabilità del sistema, non dell'hardware.

Questo cambio di prospettiva, in cui è il software a garantire l'affidabilità al posto dell'hardware, attraversa tutto il capitolo. La Tabella 1.2 mette a confronto i due approcci.

Tabella 1.2. Scalabilità verticale e orizzontale a confronto.

Aspetto	Verticale (<i>scale up</i>)	Orizzontale (<i>scale out</i>)
Approccio	una macchina più grande	più macchine
Tetto	limite fisico rigido	di fatto illimitato
Crescita del costo	superlineare nella fascia alta	circa lineare (hardware commodity)
Guasti	affidabilità dell'hardware	tolleranza ai guasti nel software
Complessità	bassa (stesso modello software)	alta (coordinamento distribuito)

1.1.4 Il guasto come evento normale

Su un cluster il guasto hardware smette di essere un caso limite. Con migliaia di dischi, statisticamente c'è sempre qualche disco che si sta rompendo; le reti si partizionano, le macchine vanno in crash, interi rack restano senza alimentazione. Un sistema progettato nell'ipotesi che i guasti siano rari, a questa scala, fallirebbe di continuo. L'infrastruttura per i big data deve trattare il guasto come un evento di routine; nella Sezione 1.6 un semplice modello probabilistico mostrerà quanto sia frequente.

Scalando in orizzontale cambiano anche le voci di costo che contano:

- **costo per byte** (storage): i dischi commodity rendono economico lo spazio grezzo, e il costo dominante diventa quello della replicazione e della durabilità;
- **costo per FLOP** (calcolo): le CPU commodity rendono economico il calcolo grezzo, e il costo dominante diventa quello del coordinamento e dello spostamento dei dati.

Entrambe le osservazioni portano alla stessa conclusione: a scala, il collo di bottiglia non è più il costo dell'hardware, ma la capacità del sistema di coordinare tante parti economiche.

1.2 Principi di progetto dei file system distribuiti

Definizione (file system distribuito). Un **file system distribuito** (*Distributed File System*, DFS) offre un'unica interfaccia di file system, sostenuta da uno storage sparso su molte macchine. Ai client i file appaiono come se stessero in un solo file system logico; in realtà il loro contenuto è suddiviso, replicato e distribuito su un cluster.

Il DFS nasconde questa complessità dietro le solite operazioni sui file (**open**, **read**, **write**, **close**). L'obiettivo è uno storage da petabyte che, per le applicazioni, sembri un normale file system, solo molto grande.

1.2.1 Obiettivi e ipotesi di progetto

I DFS per i big data sono ottimizzati per un carico preciso, e tre scelte ne determinano l'architettura.

Throughput prima della latenza. Il sistema massimizza il *throughput*, cioè la quantità totale di dati trasferiti al secondo in tutto il cluster, e tollera che la singola operazione impieghi più tempo che su un disco locale. È una scelta deliberata: l'analisi batch di petabyte ha bisogno di banda aggregata, non di tempi di risposta al millisecondo per la singola richiesta.

File enormi, letti in sequenza. I file sono di solito enormi, da gigabyte a terabyte ciascuno, e si leggono per lo più in sequenza, dall'inizio alla fine o a grandi pezzi. Le piccole letture e scritture casuali non sono il carico di riferimento. Questa ipotesi condiziona quasi ogni decisione successiva.

Write-once, read-many. Un file viene scritto una volta sola, per esempio con una serie di *append* (aggiunte in coda), poi viene chiuso e letto molte volte da job di analisi diversi. La modifica sul posto del contenuto esistente è rara o vietata. Così il sistema si risparmia un grosso onere di progetto: non deve supportare in modo efficiente aggiornamenti arbitrari sul posto.

1.2.2 Blocchi di dimensione fissa

Definizione (blocco). Un DFS non memorizza un file come un'unica unità contigua, ma lo divide in **blocchi** (*blocks*, o *chunks*) di dimensione fissa. Ogni blocco viene memorizzato, replicato e gestito in modo indipendente, e blocchi diversi dello stesso file possono stare su macchine diverse.

È la decisione strutturale più importante nel progetto di uno storage distribuito. Fissare la dimensione dei blocchi, invece di usare frammenti di lunghezza variabile, semplifica tutto il sistema:

- i **metadati**: il sistema tiene traccia di dove stanno i blocchi, non di intervalli di byte arbitrari;

- il **bilanciamento del carico**: i blocchi sono unità di lavoro intercambiabili, facili da collocare e da spostare;
- la **replicazione**: ogni blocco si replica per conto suo, in modo uniforme.

Resta da decidere quanto farli grandi (Tabella 1.3). I DFS per i big data scelgono blocchi grandi, da decine a centinaia di megabyte: l'opposto dei file system locali tradizionali, che usano blocchi di pochi kilobyte.¹

Tabella 1.3. Il compromesso sulla dimensione dei blocchi.

Blocchi	Vantaggi	Svantaggi
Piccoli (es. 4 KB)	granularità fine, poco spreco per i file piccoli	metadati enormi su larga scala
Grandi (es. 64–128 MB)	pochi metadati, I/O sequenziale efficiente	costosi per i file piccoli, bilanciamento del carico più grossolano

1.2.3 Metadati separati dai dati

Una decisione centrale è separare il sistema che sa «dove stanno le cose» da quello che conserva i byte veri e propri:

- il **servizio dei metadati** gestisce il **namespace**, cioè la struttura gerarchica di directory e file, e sa dove si trova ogni blocco;
- i **server dei dati** conservano il contenuto dei blocchi sui propri dischi locali.

Così ciascuno dei due sottosistemi si ottimizza per conto suo: il primo per le ricerche veloci, i secondi per il throughput di I/O grezzo. Il servizio dei metadati mantiene due corrispondenze: da file a blocchi (quali blocchi compongono ciascun file) e da blocchi a posizioni (quali server ospitano le repliche di ogni blocco). Le tiene soprattutto in memoria, perché su scala la velocità delle ricerche è critica per le prestazioni.

Molti DFS, GFS e HDFS compresi, affidano i metadati a un **unico nodo coordinatore**.

- **Vantaggi**: una vista del namespace semplice e globalmente consistente, e nessun bisogno di consenso distribuito a ogni modifica dei metadati.
- **Svantaggi**: il coordinatore è un potenziale punto di guasto singolo (*single point of failure*) e un potenziale collo di bottiglia, se le operazioni sui metadati crescono troppo.

Le Sezioni 1.3 e 1.4 mostrano come GFS e HDFS gestiscono questo rischio.

¹Lo svantaggio per i file piccoli non sta tanto nello spazio su disco: sia GFS sia HDFS salvano ogni blocco come un file del file system locale, che occupa solo lo spazio effettivamente scritto. Il costo sta altrove: ogni file e ogni blocco pesa sulla memoria del nodo che gestisce i metadati, e un file piccolo sta in pochissimi blocchi, quindi le poche macchine che lo ospitano diventano punti caldi (*hot spot*) se molti client lo leggono insieme.

Il coordinatore, poi, non vede passare i dati. Il client gli chiede solo dove si trova un blocco, riceve l'elenco delle posizioni e scambia i dati direttamente con i server dei dati (lo schema concreto è in Figura 1.1). Le interrogazioni dei metadati, piccole e frequenti, restano così separate dai trasferimenti dei dati, grandi e pesanti in banda, e il coordinatore non diventa un collo di bottiglia.

1.2.4 Topologia di rete e replicazione

Gli storage distribuiti conoscono la topologia fisica della rete, non solo l'identità logica delle macchine. Le macchine sono raggruppate in **rack**, ciascuno con il proprio switch (*top-of-rack switch*), e la banda all'interno di un rack è di solito molto più alta di quella tra rack diversi. Le decisioni di posizionamento e di scheduling tengono conto di questa gerarchia: macchina, rack, datacenter. Servirà per collocare le repliche (Sezione 1.5).

Su una singola macchina, contro il guasto di un disco basta il RAID. Su un cluster non più: il RAID protegge dal guasto di un disco, non da quello di una macchina, di un rack o della rete, e con migliaia di macchine il numero atteso di guasti in ogni istante non è zero. È il DFS stesso a dover replicare i dati su macchine diverse, e possibilmente su rack diversi. Qui la **replicazione** non è un'ottimizzazione: è il meccanismo principale di durabilità.

1.3 Google File System

Il **Google File System** (GFS), pubblicato nel 2003,² è stato progettato per i carichi interni di Google: memorizzare ed elaborare enormi quantità di pagine web, di log e di file di indice, su migliaia di macchine commodity in cui il guasto è la norma. È ottimizzato per file enormi, letture sequenziali e scritture fatte soprattutto di append, non per la semantica POSIX di un file system generico. È il diretto antenato di HDFS.

1.3.1 Architettura

Un cluster GFS ha un solo **master**, che coordina, e molti **chunkserver**, che memorizzano i dati e li servono direttamente ai client (Figura 1.1). Tra master e chunkserver passano solo informazioni di controllo; i dati viaggiano direttamente tra client e chunkserver, senza passare dal master.

Il master. È l'unico nodo che gestisce i metadati del cluster:

- mantiene il namespace del file system, cioè la struttura delle directory e la corrispondenza tra file e chunk;
- mantiene la mappa delle posizioni: quali chunkserver ospitano quali chunk;
- assegna i *lease* per le scritture (Sezione 1.3.3);
- coordina la creazione, la replicazione e la *garbage collection* dei chunk.

²S. Ghemawat, H. Gobioff, S.-T. Leung, *The Google File System*, SOSP 2003.

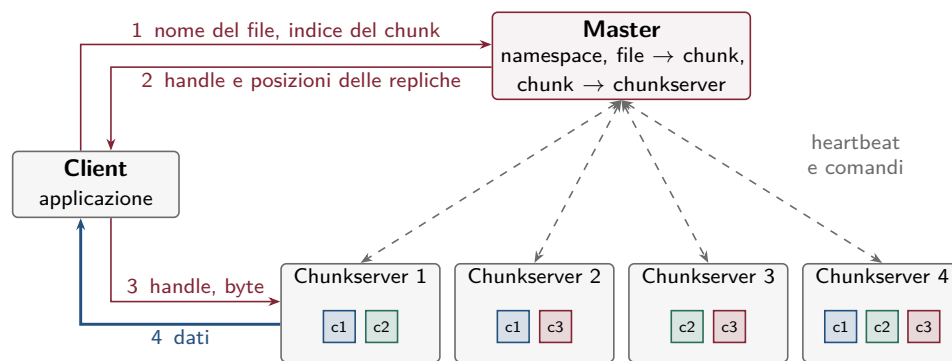


Figura 1.1. Architettura di GFS e percorso di una lettura. In rosso i messaggi di controllo, in blu il flusso dei dati, tratteggiati heartbeat e comandi. Ogni chunk (c1, c2, c3) ha tre repliche su chunkserver diversi. HDFS ha la stessa struttura, con NameNode e DataNode al posto di master e chunkserver.

Tutti questi metadati stanno in memoria, così il master risponde molto in fretta. Le posizioni dei chunk, anzi, non vengono nemmeno salvate su disco: all'avvio il master le ricostruisce interrogando i chunkserver. Il prezzo è un limite: la dimensione complessiva del namespace è vincolata dalla memoria del master.

La scelta di un master unico, invece di un cluster distribuito di metadati, è deliberata. Semplifica drasticamente il progetto, perché le modifiche dei metadati non richiedono consenso distribuito, e dà una vista unica e consistente delle decisioni di posizionamento dei chunk. I rischi sono quelli di ogni coordinatore unico: punto di guasto singolo, mitigato come si vedrà in Sezione 1.3.5, e possibile collo di bottiglia se le operazioni sui metadati crescono troppo.

I chunkserver. Sono i cavalli da lavoro di GFS. Ogni chunkserver salva i chunk come normali file sul proprio file system locale (Linux), serve direttamente ai client le letture e le scritture dei chunk che possiede e comunica periodicamente al master il proprio inventario.

Chunk da 64 MB. GFS usa chunk molto grandi, di 64 MB, ben più dei blocchi di un file system locale:

- meno chunk per file significano meno metadati sul master;
- una lettura lunga richiede poche ricerche di chunk, quindi poche interazioni tra client e master;
- ogni chunk si legge con un trasferimento grande, continuo e sequenziale.

Lo svantaggio, già visto, riguarda i file piccoli: un compromesso accettato, dato il carico a cui GFS è destinato.

Replicazione. Per default ogni chunk è replicato 3 volte, su chunkserver diversi. Le repliche proteggono dal guasto di un singolo chunkserver o disco e sono collocate tenendo conto dei rack (Sezione 1.5). Il fattore di replicazione si può configurare file per file, così i dati più critici possono avere più copie.

Heartbeat. Master e chunkserver dialogano con gli **heartbeat** (letteralmente «battiti cardiaci»): messaggi periodici con cui ogni chunkserver segnala di essere vivo e riporta quali chunk possiede. Servono a più cose:

- il master sa quali chunkserver sono attivi e che cosa contengono, senza dover salvare su disco le posizioni dei chunk;
- confrontando gli inventari, riconosce i chunk sotto-replicati, dopo un guasto, e quelli sovra-replicati, dopo un recupero;
- nella risposta all'heartbeat include i propri comandi, come «replica questo chunk altrove» o «cancella quest'altro».

1.3.2 Lettura

Una lettura procede così (i numeri sono quelli della Figura 1.1):

1. dall'offset richiesto il client calcola l'indice del chunk (l'offset diviso per 64 MB, arrotondato per difetto) e chiede al master il *chunk handle*, cioè l'identificativo del chunk, e l'elenco dei chunkserver che lo ospitano;
2. il master risponde con handle e posizioni, che il client tiene in cache per un breve periodo;
3. il client contatta direttamente la replica più vicina, scavalcando il master;
4. il chunkserver trasmette i dati richiesti direttamente al client.

Il master interviene solo nei primi due passi, cioè sui metadati: il grosso dei dati passa direttamente tra client e chunkserver.

1.3.3 Scrittura e lease

Quando più client modificano lo stesso chunk, tutte le repliche devono applicare le modifiche nello stesso ordine. GFS non fa decidere l'ordine al master per ogni scrittura, e non usa un protocollo di consenso: delega la decisione a una delle repliche, tramite un lease.

Definizione (lease). Un **lease** (letteralmente «locazione») è una concessione a tempo con cui il master designa, per un certo chunk, una delle repliche come **primaria**. Finché il lease è valido, la primaria decide l'ordine seriale di tutte le modifiche (*mutations*) concorrenti di quel chunk; le altre repliche, le **secondarie**, le applicano in quell'ordine.

Il lease ha una scadenza, ma può essere prolungato finché la primaria sta gestendo delle modifiche. Se la primaria diventa irraggiungibile, il master aspetta che il vecchio lease scada e poi ne concede uno nuovo a un'altra replica. In questo modo il master non deve decidere l'ordine di ogni singola scrittura: lo delega alla replica che detiene il lease.

La scrittura procede in cinque passi (Figura 1.2):

1. il client chiede al master quali chunkserver ospitano le repliche del chunk e quale di esse detiene il lease, cioè è la primaria;

2. il client invia (*push*) i dati a tutte le repliche, in qualsiasi ordine; le repliche li tengono temporaneamente in un buffer;
3. il client manda la richiesta di scrittura alla primaria;
4. la primaria assegna alla scrittura un numero d'ordine e la inoltra alle secondarie, che la applicano in quell'ordine;
5. le secondarie confermano alla primaria, e la primaria risponde al client quando tutte le repliche hanno confermato.

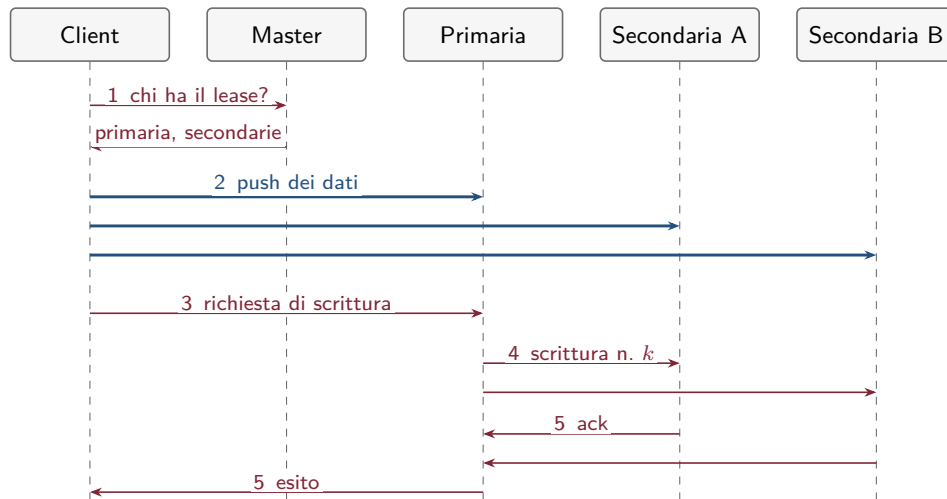


Figura 1.2. Scrittura in GFS. Il flusso dei dati (in blu) è separato dal flusso di controllo (in rosso): i dati arrivano prima a tutte le repliche, poi la primaria decide in che ordine applicarli.

Il flusso dei dati (il push) e il flusso di controllo (l'ordine delle scritture) sono separati di proposito, per le prestazioni: il trasferimento pesante non aspetta le decisioni sull'ordine, e l'ordine si decide con messaggi piccoli.

Consistenza. Poiché tutte le repliche applicano le modifiche nello stesso ordine, quello dettato dalla primaria, le repliche convergono allo stesso contenuto del chunk. Le garanzie complessive restano però rilassate: GFS non offre la semantica POSIX stretta, e con più scrittori concorrenti una regione del file può contenere, mescolati tra loro, frammenti di scritture diverse.³ È un compromesso accettato in cambio delle prestazioni.

1.3.4 Cancellazione e garbage collection

GFS non libera subito lo spazio di un file cancellato. Il file viene rinominato con un nome nascosto e conservato per qualche giorno; lo spazio dei suoi chunk viene

³Le slide parlano di interleaving «*defined but undefined*», che è un refuso. Nella terminologia del paper una regione del file è *consistent* se tutti i client vedono gli stessi dati, da qualunque replica leggano, ed è *defined* se in più contiene per intero ciò che una scrittura ha scritto. Dopo scritture concorrenti andate a buon fine la regione è *consistent but undefined*: tutti vedono gli stessi dati, che però possono mescolare frammenti di più scritture.

recuperato più tardi, in modo pigro (*lazy*), da un processo di **garbage collection** che lavora durante le normali comunicazioni tra master e chunkserver. La cancellazione diventa più semplice e robusta: una cancellazione accidentale si può annullare entro il periodo di conservazione, e non serve un'operazione sincrona su tutto il cluster.

1.3.5 Tolleranza ai guasti del master

Il master tiene i metadati in memoria, ma deve sopravvivere ai crash:

- ogni modifica del namespace viene aggiunta in coda a un **operation log** persistente, replicato su più macchine;
- periodicamente il master scrive un **checkpoint**, un'istantanea compatta del proprio stato, per non dover rieseguire da capo, al riavvio, un log sempre più lungo;
- al riavvio carica l'ultimo checkpoint e riesegue le voci del log successive.

GFS usa anche degli **shadow master**: repliche in sola lettura dello stato del master, che applicano continuamente lo stesso operation log. Possono rispondere alle interrogazioni di sola lettura sui metadati, alleggerendo il master, ma se il master cade non ne prendono automaticamente il posto: la sostituzione richiede una procedura separata di recupero o di promozione.

1.3.6 Limiti

Il master unico, per quanto semplice, ha limiti noti:

- **scalabilità dei metadati**: l'intero namespace deve stare nella memoria di una macchina;
- **collo di bottiglia**: un ritmo molto alto di operazioni sui metadati, per esempio con moltissimi file piccoli, può sovraccaricare il master;
- **failover non istantaneo**: anche con checkpoint e log, riprendersi da un crash del master richiede tempo.

Sono proprio questi limiti a motivare le evoluzioni successive, come la Federation di HDFS.

1.4 HDFS

L'**Hadoop Distributed File System** (HDFS) è lo storage open source ispirato direttamente al paper di GFS, nato nel progetto Apache Hadoop per l'elaborazione batch su larga scala, all'inizio proprio per MapReduce (Capitolo 2). Ne adotta l'architettura: un nodo che gestisce i metadati, molti nodi che memorizzano i dati, blocchi grandi, replicazione. Cambia la terminologia, ma la filosofia di progetto è quasi identica. La Tabella 1.4 mette a confronto i due sistemi; nella riga dei checkpoint l'analogia è solo parziale.⁴

⁴Lo shadow master di GFS risponde alle letture sui metadati; il Secondary NameNode di HDFS non serve alcuna richiesta, ma si limita a fondere FsImage ed EditLog in un nuovo checkpoint.

Tabella 1.4. GFS e HDFS: terminologia e differenze principali.

Concetto	GFS	HDFS
Nodo dei metadati	master	NameNode
Nodo dei dati	chunkserver	DataNode
Unità di memorizzazione	chunk	blocco (<i>block</i>)
Dimensione dell'unità	64 MB, fissa	128 MB per default, configurabile (di solito 128–256 MB)
Persistenza dei metadati	checkpoint + operation log	FsImage + EditLog
Aiuto per i checkpoint	shadow master	Secondary NameNode
Scrittura	push a tutte le repliche, ordine deciso dalla primaria con il lease	pipeline di DataNode

1.4.1 NameNode e DataNode

Il **NameNode** è l'equivalente del master di GFS: mantiene il namespace (directory, file e permessi) e le corrispondenze tra file e blocchi e tra blocchi e DataNode, coordina replicazione, cancellazione e ribilanciamento dei blocchi, e tiene tutti questi metadati in memoria. I **DataNode** sono l'equivalente dei chunkserver: salvano i blocchi come file sul proprio file system locale (per esempio ext4 o XFS) e servono direttamente ai client letture e scritture.

I blocchi di HDFS sono di 128 MB per default, e si possono configurare (valori comuni vanno da 128 a 256 MB), contro i 64 MB fissi di GFS. La motivazione è la stessa: pochi metadati e trasferimenti grandi e sequenziali.

Heartbeat e block report. In GFS l'heartbeat portava anche l'inventario dei chunk; HDFS separa i due compiti in due messaggi periodici:

- gli **heartbeat**, frequenti (ogni pochi secondi), sono un semplice segnale di vita: «sono attivo»;
- i **block report**, meno frequenti, contengono l'elenco completo dei blocchi presenti sul DataNode.

Insieme danno al NameNode una vista accurata, quasi in tempo reale, della distribuzione dei blocchi e della salute dei nodi.

1.4.2 FsImage, EditLog e Secondary NameNode

Il NameNode rende persistenti i metadati con due strutture complementari, equivalenti al checkpoint e all'operation log di GFS:

- la **FsImage**, un'istantanea completa del namespace in un certo momento;

- l'**EditLog**, un log in sola aggiunta (*append-only*) di tutte le modifiche del namespace (creazioni, cancellazioni, rinomine...) successive all'ultima FsImage.

All'avvio il NameNode carica in memoria la FsImage più recente e riesegue tutte le voci dell'EditLog registrate dopo di essa; lo stato che ottiene è esattamente quello di prima del riavvio. Poco dopo si prende di solito una nuova FsImage, per limitare il tempo di riesecuzione del log al riavvio successivo.

Il Secondary NameNode non è un NameNode di riserva. Il nome inganna: il **Secondary NameNode** non è una replica pronta a subentrare (*hot standby*). Il suo compito è fondere periodicamente FsImage ed EditLog in un nuovo checkpoint e rimandarlo al NameNode, togliendo questo lavoro, potenzialmente costoso, dal percorso critico del NameNode. Per avere un vero failover serve la configurazione ad alta affidabilità.

1.4.3 Alta affidabilità e Federation

Per eliminare il punto di guasto singolo, le installazioni di produzione usano coppie di NameNode in **High Availability** (HA):

- un NameNode *Active* gestisce tutte le richieste dei client;
- uno o più NameNode *Standby* restano sincronizzati, pronti a subentrare;
- un edit log condiviso e consistente, spesso affidato a un servizio di logging separato basato sul consenso, tiene aggiornati gli Standby;
- un meccanismo di failover automatico rileva il guasto dell'Active e promuove uno Standby.

Contro il collo di bottiglia dei metadati, lo stesso limite visto per GFS, HDFS offre la **Federation**: più NameNode indipendenti, ciascuno responsabile di una porzione distinta del namespace. I DataNode si registrano presso tutti i NameNode della federazione e ospitano blocchi per ciascuno di essi. Ogni NameNode ha ancora il proprio limite di memoria, ma la capacità complessiva del namespace cresce con il numero di NameNode.

1.4.4 Lettura e scrittura

La lettura segue lo schema di GFS. Il client chiede al NameNode le posizioni dei blocchi del file; il NameNode restituisce gli identificativi dei blocchi e, per ciascuno, l'elenco dei DataNode che lo ospitano, ordinato quando possibile per vicinanza al client; il client si collega al DataNode disponibile più vicino, che gli trasmette i dati senza coinvolgere il NameNode.

La scrittura invece usa una **pipeline** di repliche (Figura 1.3):

1. il client chiede al NameNode di creare un nuovo file; il NameNode controlla i permessi e alloca il primo blocco;
2. il NameNode restituisce l'elenco dei DataNode che formano la pipeline di replicazione del blocco;
3. il client invia il flusso dei dati al primo DataNode della pipeline;

4. ogni DataNode inoltra i dati al successivo, in sequenza;
5. le conferme (*ack*) risalgono la pipeline quando tutti i DataNode hanno reso persistenti i dati.

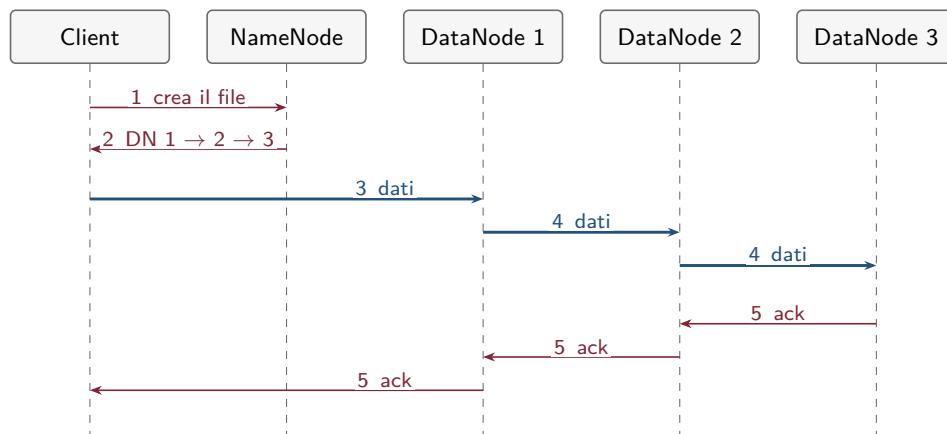


Figura 1.3. Scrittura in HDFS: i dati attraversano una sola volta la catena di DataNode, e le conferme la risalgono all'indietro.

È qui la differenza principale con GFS: in HDFS i dati attraversano una volta sola una catena di nodi, mentre in GFS il client li spinge a ogni replica e poi la scrittura si coordina con la primaria che detiene il lease.⁵ Le altre differenze sono minori: la persistenza dei metadati con FsImage ed EditLog è l'analogo del checkpoint e dell'operation log; alta affidabilità e Federation in HDFS sono estensioni esplicite, da configurare a parte, mentre il paper di GFS descrive gli shadow master, e una soluzione completa di alta affidabilità e federazione è maturata solo dopo, nella pratica industriale.

Solo append. Coerentemente con l'ipotesi write-once-read-many, HDFS permette di aggiungere dati in coda a un file esistente, ma non di modificarlo sul posto: nel modello classico di HDFS le scritture casuali nel mezzo di un file non sono supportate. È lo stesso compromesso di GFS, con lo stesso scopo: semplificare consistenza e replicazione.

1.4.5 I blocchi e il calcolo batch

Visti due sistemi concreti, l'astrazione del blocco mostra tutti i suoi vantaggi. I blocchi di dimensione fissa rendono gestibili, per un unico master o NameNode in memoria, i metadati di file da petabyte; la dimensione uniforme semplifica replicazione, bilanciamento e ri-replicazione dopo i guasti; la stessa astrazione è alla base di praticamente ogni storage distribuito venuto dopo GFS e HDFS.

Soprattutto, i blocchi si prestano al calcolo batch:

⁵Nel paper di GFS, in realtà, anche il push dei dati avviene in catena: ogni chunkserver inoltra i dati al successivo, scelto tra i più vicini in rete. Ciò che distingue davvero GFS è la separazione tra flusso dei dati e flusso di controllo, con l'ordine delle scritture deciso dalla primaria.

- ogni blocco si può elaborare in modo in gran parte indipendente dagli altri, quindi molti blocchi si elaborano in parallelo;
- le posizioni dei blocchi sono visibili agli scheduler, e questo rende possibile la località dei dati (Sezione 1.5).

È esattamente la struttura che sfrutta MapReduce: un task di map per blocco, eseguito vicino ai dati del blocco.

1.5 Località dei dati e replicazione

1.5.1 Spostare il calcolo, non i dati

Definizione (località dei dati). Il principio di **località dei dati** (*data locality*) dice che, su scala big data, portare il calcolo dove i dati già risiedono costa molto meno che portare i dati dove si fa il calcolo. Lo scheduler esegue quindi ogni elaborazione sulla macchina che ha già i dati che le servono, o su una vicina.

Il motivo è che la banda di rete tra macchine è di ordini di grandezza inferiore a quella dei dischi e della memoria locali, e spostare in rete terabyte di dati per ogni job sarebbe proibitivamente lento e costoso. Il principio, centrale in MapReduce, nasce dallo strato di storage: GFS e HDFS dicono dove si trova fisicamente ogni blocco, lo scheduler può collocare il calcolo su quella macchina, e in rete viaggiano solo i risultati, molto più piccoli dei dati grezzi.

Tabella 1.5. Bande indicative in un cluster tipico.

Risorsa	Banda indicativa
Lettura da disco locale	da centinaia di MB/s a qualche GB/s
Rete nello stesso rack	circa 1–10 Gb/s, condivisi da tutto il rack
Rete tra rack	meno, condivisa sulla dorsale (<i>backbone</i>) del datacenter

La Tabella 1.5 dà gli ordini di grandezza. Nel leggerla conviene notare due cose: la banda di rete è in gigabit al secondo (10 Gb/s sono circa 1,25 GB/s), e soprattutto è condivisa da tutte le macchine del rack, mentre ogni macchina ha i propri dischi. L'I/O locale è veloce e «gratuito»; l'I/O di rete, specie tra rack, è la risorsa scarsa che la località dei dati serve a risparmiare.

1.5.2 Posizionamento delle repliche

Dove collocare le repliche di un blocco non si decide a caso: bisogna bilanciare due obiettivi in conflitto.

- **Durabilità:** le repliche vanno sparse su domini di guasto diversi, cioè su macchine diverse e possibilmente su rack diversi.
- **Efficienza:** sparpagiarle troppo aumenta il traffico tra rack durante le scritture.

Il compromesso comune è il posizionamento **rack-aware**: alcune repliche nello stesso rack, e almeno una in un rack diverso. Con fattore 3, due repliche condividono un rack, dove la replicazione è veloce, e la terza sta in un altro, così il blocco sopravvive anche alla perdita di un intero rack, per esempio per un guasto dell'alimentazione o dello switch che il rack condivide.

La politica di default di HDFS, con fattore 3, è la seguente (Figura 1.4):

1. la prima replica va sul DataNode su cui scrive il client, o su un nodo vicino;
2. la seconda va su un nodo di un rack diverso da quello della prima;
3. la terza va su un altro nodo dello stesso rack della seconda.

La durabilità tra rack è garantita, perché la prima replica sta in un rack e le altre due nell'altro; il traffico tra rack in scrittura resta limitato, perché le repliche 2 e 3 condividono un solo passaggio da un rack all'altro.

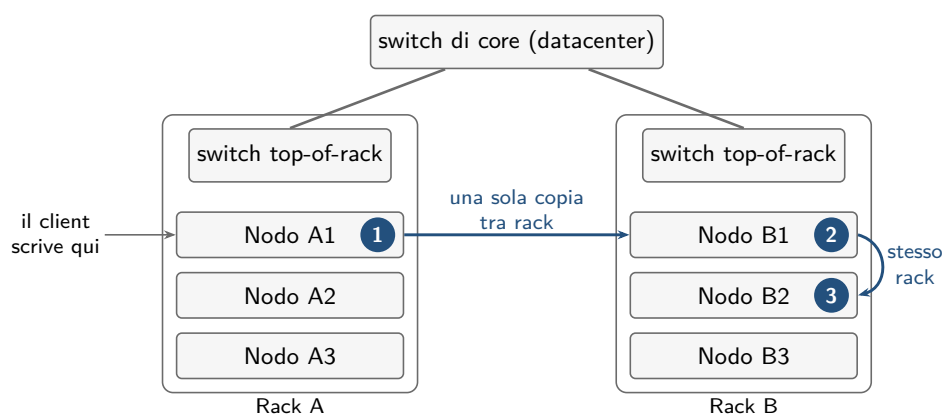


Figura 1.4. Posizionamento di default delle repliche in HDFS, con fattore 3. Una sola copia attraversa la rete tra i rack; se si guasta un rack intero, nell'altro resta almeno una replica.

HDFS però non deduce da solo la topologia della rete: è l'amministratore a doverla configurare, con uno script o una tabella che associa ogni DataNode al proprio rack. Senza questa configurazione HDFS tratta tutti i nodi come se fossero in un unico rack, e perde i benefici del posizionamento tra rack. È un passo di installazione che nei cluster messi su in modo ingenuo viene spesso dimenticato.

1.5.3 Fattore di replicazione

Definizione (fattore di replicazione). Il **fattore di replicazione** r è il numero di copie mantenute per ogni blocco.

Aumentarlo migliora la durabilità, ma costa in spazio (Tabella 1.6) e in scrittura: ogni scrittura deve raggiungere tutte le repliche prima di essere confermata, come si è visto per GFS e per HDFS, quindi più repliche significano più traffico di rete e più tempo prima che una scrittura sia considerata durevole. Fattori più alti del default si usano in modo selettivo, per dati particolarmente critici o letti molto spesso.

Tabella 1.6. Fattore di replicazione: spazio, guasti tollerati e probabilità di perdita, con $p = 0,02$ (Esempio sotto).

Fattore r	Spazio occupato	Guasti tollerati	$P(\text{perdita}) \approx p^r$
1 (nessuna replica)	$1\times$	nessuno	0,02
2	$2\times$	1	0,0004
3 (default)	$3\times$	2 simultanei	0,000008

Esempio (probabilità di perdere un chunk). Supponiamo che ogni nodo si guasti con probabilità annua $p = 0,02$, indipendentemente dagli altri. Un chunk replicato su r nodi indipendenti va perso solo se tutte le repliche si guastano insieme, nella stessa breve finestra di tempo, il che succede con probabilità

$$P(\text{perdita}) \approx p^r.$$

Con $r = 1, 2, 3$ si ottengono 0,02, 0,0004 e 0,000008 (Tabella 1.6): ogni replica in più divide la probabilità di perdita per $1/p = 50$, quasi due ordini di grandezza.⁶ Il calcolo però vale solo se i guasti sono indipendenti: è proprio l'indipendenza che il posizionamento rack-aware cerca di garantire.

1.5.4 Domini di guasto e bilanciamento

La replicazione deve tenere conto dei guasti correlati, che colpiscono insieme ambiti via via più ampi, i **domini di guasto** (*failure domains*):

- il **disco**: si perdono le copie che stavano su quel disco;
- il **nodo**: si perdono tutti i chunk della macchina;
- il **rack**: si perdono tutti i nodi che condividono lo switch o l'alimentazione;
- il **datacenter**: si perde un'intera struttura, per un blackout, un guasto di rete o un disastro.

Distribuire le repliche su questi domini protegge da guasti di scala via via più grande.

Oltre a replicare, il sistema cerca di bilanciare l'occupazione dello spazio, per non sovraccaricare alcuni nodi mentre altri restano semivuoti: sia il posizionamento dei nuovi chunk sia la ri-replicazione tengono conto dell'utilizzo attuale dei dischi. Un cluster ben bilanciato evita i punti caldi, che diventerebbero colli di bottiglia o punti di guasto precoce.

1.6 Gestione dei guasti

Con migliaia di dischi e di macchine i guasti sono un processo continuo, di sottofondo, non incidenti isolati; heartbeat, replicazione, lease e checkpoint esistono tutti per

⁶Il modello è volutamente grossolano: p è una probabilità annua, mentre la perdita richiede che le repliche si guastino nella stessa breve finestra di tempo. I numeri mostrano l'effetto dell'esponente r , non stimano il rischio reale.

renderli sopportabili. La qualità di uno storage distribuito si misura soprattutto da come degrada e si riprende, non da quanto raramente si guasta. Questa sezione quantifica prima quanti guasti aspettarsi, poi raccoglie i meccanismi per rilevarli e per rimediare.

1.6.1 Quanti guasti aspettarsi

I guasti dei dischi si misurano con il **tasso annuo di guasto** (*Annualized Failure Rate*, AFR), cioè la probabilità che un disco si guasti nel corso di un anno. Gli studi su larga scala sui dischi commodity riportano valori tra l'1% e il 5% circa, che variano con l'età, il modello e le condizioni d'uso. Per un disco solo è poco; per un cluster no.

Consideriamo un cluster con

$$N = 10\,000 \text{ dischi}, \quad \text{AFR} = 2\% = 0,02,$$

e sia p la probabilità che un certo disco si guasti durante una settimana. Supponendo per semplicità che i guasti siano distribuiti in modo uniforme nel corso dell'anno,

$$p \approx \frac{0,02}{52} \approx 0,0003846.$$

Il numero di guasti è una variabile aleatoria. Per il disco i si definisce la variabile indicatrice

$$X_i = \begin{cases} 1 & \text{se il disco } i \text{ si guasta durante la settimana,} \\ 0 & \text{altrimenti,} \end{cases}$$

cioè $X_i \sim \text{Bernoulli}(p)$. Il numero totale di guasti nella settimana è a sua volta una variabile aleatoria, $X = \sum_{i=1}^N X_i$. Se i guasti dei dischi sono indipendenti e identicamente distribuiti, X è la somma di N variabili di Bernoulli i.i.d., quindi ha distribuzione binomiale:

$$X \sim \text{Binomial}(N, p), \quad N = 10\,000, \quad p \approx 0,0003846.$$

Probabilità di k guasti. La probabilità di osservare esattamente k guasti è

$$P(X = k) = \binom{N}{k} p^k (1 - p)^{N-k}.$$

Per esempio, la probabilità di esattamente 4 guasti in una settimana è

$$P(X = 4) = \binom{10\,000}{4} (0,0003846)^4 (1 - 0,0003846)^{9996} \approx 0,195,$$

quasi il 20%.⁷

⁷Le slide riportano 19,4%: il valore è 0,1948, che arrotondato fa 19,5%.

Valore atteso e varianza. Per una binomiale $\mathbb{E}[X] = Np$, quindi

$$\mathbb{E}[X] = 10\,000 \times \frac{0,02}{52} \approx 3,85 :$$

in media il cluster perde quasi 4 dischi a settimana. Il valore atteso però non è un valore che X possa assumere: $X \in \{0, 1, 2, 3, \dots\}$ è un intero, mentre 3,85 è la media sul lungo periodo. Quanto X oscilla attorno alla media lo dice la varianza, che per una binomiale è $\text{Var}(X) = Np(1-p)$:

$$\text{Var}(X) = 10\,000 \cdot 0,0003846 \cdot (1 - 0,0003846) \approx 3,84, \quad \sigma_X = \sqrt{\text{Var}(X)} \approx 1,96.$$

A grandi linee, quindi, $X \approx 3,85 \pm 1,96$: il numero di guasti settimanali fluttua in modo naturale attorno al valore atteso.

Almeno un guasto. Qual è la probabilità che in una settimana si guasti almeno un disco? Invece di sommare le probabilità di $X = 1, 2, \dots$, conviene passare al complemento, cioè calcolare la probabilità che non se ne guasti nessuno:

$$P(X \geq 1) = 1 - P(X = 0) = 1 - (1 - p)^N = 1 - (1 - 0,0003846)^{10\,000} \approx 0,979.$$

Con queste ipotesi semplificate, un cluster di 10 000 dischi ha almeno un guasto nel 98% circa delle settimane. È l'effetto di scala che giustifica tutta la filosofia del «guasto normale»: un tasso annuo del 2% sembra piccolo per un disco, ma su 10 000 dischi diventa un flusso regolare di guasti, circa 4 a settimana, non all'anno.

1.6.2 Tipi di guasto

Non tutti i guasti sono uguali, e il sistema deve gestirne di diversi tipi:

- **transitori**: brevi interruzioni di rete o pause momentanee di un processo, per esempio per la garbage collection; il nodo sta bene, ma per un po' non risponde;
- **permanenti**: un disco che muore, un guasto hardware, un nodo dismesso; il nodo o i dati sono persi per sempre;
- **correlati**: guasti che colpiscono più macchine insieme, come la perdita di alimentazione di un rack o il guasto del suo switch.

1.6.3 Rilevamento: heartbeat e timeout

GFS e HDFS rilevano i guasti con lo stesso meccanismo: ogni nodo dei dati manda heartbeat periodici al nodo dei metadati e, se non arriva nessun heartbeat entro un certo *timeout*, il nodo viene considerato morto. È intrinsecamente un compromesso, e ricorre spesso nei sistemi distribuiti (Tabella 1.7): un timeout troppo breve rischia falsi positivi, cioè nodi sani ma lenti dati per morti, mentre uno troppo lungo ritarda la scoperta dei guasti veri. Non esiste un valore giusto in assoluto: si regola sull'affidabilità della rete e sul carico del cluster specifico.

Tabella 1.7. Il compromesso sul timeout degli heartbeat.

Timeout	Rilevamento	Rischio
Breve	rapido	falsi positivi su nodi lenti ma vivi; ri-replicazioni inutili
Lungo	meno falsi positivi	risposta lenta ai guasti veri; finestra più lunga con ridondanza ridotta

1.6.4 Recupero

Guasto di un nodo dei dati. Quando un chunkserver o un DataNode viene dato per morto, il recupero segue sempre lo stesso schema:

1. il nodo dei metadati rileva il guasto dagli heartbeat mancati;
2. individua tutti i chunk o blocchi che stavano su quel nodo e li segna come sotto-replicati;
3. pianifica la ri-replicazione di ciascuno, copiandolo da una replica superstite su un nodo sano; la copia avviene in background, con la priorità di ripristinare la ridondanza il più in fretta che la rete e i dischi permettono;
4. alla fine il sistema torna al fattore di replicazione pieno, in automatico, senza intervento manuale.

Guasto del nodo dei metadati. Recuperare il master o il NameNode è più delicato, perché contiene l'unica vista autorevole del namespace. Senza alta affidabilità, il cluster non accetta nuove operazioni finché il nodo non viene riavviato a mano, ricaricando checkpoint e log come visto nella Sezione 1.3.5. Con l'alta affidabilità, uno Standby viene promosso ad Active e riparte con un'interruzione minima, grazie all'edit log condiviso e sincronizzato di continuo. Per questo le installazioni di produzione usano quasi sempre l'alta affidabilità, nonostante la complessità in più.

Corruzione silenziosa e checksum. Non tutti i guasti sono crash evidenti: un disco può restituire bit leggermente corrotti senza segnalare alcun errore. GFS e HDFS salvano un **checksum** insieme a ogni chunk o blocco e lo verificano a ogni lettura: se non corrisponde, i dati sono corrotti. Una corruzione scoperta fa scattare lo stesso meccanismo di ri-replicazione: si legge da un'altra replica, presumibilmente integra, e si ricrea quella corrotta.

Auto-riparazione. Heartbeat, block report e checksum insieme rendono il sistema capace di ripararsi da solo, in modo continuo: i blocchi sotto-replicati per un guasto tornano al fattore pieno, i blocchi corrotti vengono sostituiti a partire da repliche integre. Per i guasti singoli e di routine non serve alcun intervento umano; gli operatori servono solo per eventi su larga scala o insoliti.

Esempio (un chunkserver muore durante una lettura). Un caso concreto, dall'inizio alla fine:

1. un client sta leggendo un chunk dal chunkserver A, che va in crash a metà trasferimento;
2. la richiesta del client va in timeout o restituisce un errore;
3. il client chiede di nuovo al master le posizioni delle altre repliche dello stesso chunk;
4. il client ripete la lettura da un altro chunkserver, B o C: per l'applicazione la lettura riesce, in modo trasparente per l'utente finale;
5. separatamente, il master si accorge degli heartbeat mancati di A e pianifica la ri-replicazione di tutti i suoi chunk altrove.

I passi dal 2 al 4 riguardano il client; il passo 5 avviene del tutto in background.

1.7 Dallo storage al calcolo

GFS e HDFS sono due realizzazioni della stessa architettura, una proprietaria e l'altra open source, divenuta lo standard dell'industria. Tre pilastri la reggono:

1. **scalabilità orizzontale su hardware commodity**: tante macchine economiche e inaffidabili invece di poche costose e affidabili;
2. **storage a blocchi replicati**: blocchi di dimensione fissa, replicati su domini di guasto diversi, che danno insieme parallelismo e durabilità;
3. **metadati separati e tolleranti ai guasti**: un unico nodo coordinatore (master o NameNode) sa dove stanno i dati, mentre i dati viaggiano direttamente tra i client e i nodi che li conservano.

Preso da solo, tutto questo è «solo» un sistema di storage; il suo significato si vede quando lo si affianca al calcolo. I blocchi permettono l'elaborazione in parallelo, un task per blocco; le posizioni esposte dei blocchi permettono la località dei dati; replicazione e tolleranza ai guasti nello strato di storage permettono allo strato di calcolo di dare per scontato uno storage affidabile, e di semplificarsi di conseguenza. È così che funzionerà MapReduce: legge l'input come insieme di blocchi HDFS (o chunk GFS), esegue ogni task di map sul nodo che ha già il suo blocco, o vicino, e riscrive l'output nello stesso file system distribuito, replicato come qualunque altro dato. MapReduce non reinventa lo storage: poggia su tutto ciò che si è visto in questo capitolo.

L'architettura ha però limiti noti, che motivano gli sviluppi successivi del corso:

- è ottimizzata per file grandi e accesso sequenziale, e si adatta male a moltissimi file piccoli o ad accessi casuali;
- il nodo unico dei metadati, anche con alta affidabilità e Federation, limita la scalabilità del namespace;
- la semantica write-once-read-many è restrittiva per i carichi che aggiornano spesso i dati sul posto.

Le lezioni successive (Spark, streaming, architetture *lakehouse*) ne affrontano diversi direttamente. La prossima costruisce il primo strato di calcolo sopra questo storage: MapReduce.

In sintesi

- Scale-out: tante macchine commodity invece di una sempre più grande. Il costo cresce circa linearmente e non c'è un tetto fisico, ma il guasto diventa normale e l'affidabilità passa al software.
- Un DFS presenta un unico file system logico sopra un cluster; è pensato per il throughput, per file enormi letti in sequenza e per la semantica write-once-read-many.
- I file sono divisi in blocchi grandi e di dimensione fissa (64 MB in GFS, 128 MB in HDFS), replicati in modo indipendente, 3 volte per default.
- I metadati (namespace e posizioni dei blocchi) stanno in memoria su un unico coordinatore, master o NameNode, e sopravvivono ai crash grazie a log e checkpoint; i dati viaggiano direttamente tra client e server dei dati.
- In GFS l'ordine delle scritture lo decide la replica primaria che detiene il lease, e il flusso dei dati è separato da quello di controllo; HDFS scrive con una pipeline di DataNode e aggiunge alta affidabilità e Federation.
- Località dei dati: si porta il calcolo dove stanno i dati, perché la rete, soprattutto tra rack, è la risorsa scarsa. In HDFS una replica sta nel rack di chi scrive e due in un altro rack.
- I guasti si rilevano con heartbeat e timeout e si riparano da soli con la ri-replicazione; i checksum scoprono la corruzione silenziosa. Con 10 000 dischi e AFR del 2%, $X \sim \text{Binomial}(N, p)$: in media 3,85 guasti a settimana, e almeno uno nel 98% delle settimane.

Capitolo 2

Il modello di esecuzione MapReduce

Il capitolo precedente ha dato uno storage da petabyte, GFS o HDFS, ma lo storage da solo non calcola nulla: i dati stanno in blocchi replicati su migliaia di macchine, e qualcuno deve pur elaborarli. Scrivere codice distribuito su misura per ogni analisi è lento, soggetto a errori e difficile da rendere tollerante ai guasti. **MapReduce** è un modello di programmazione in cui il programmatore scrive codice dall'aspetto sequenziale, mentre il sistema sottostante si occupa di parallelismo, scheduling e guasti.¹ MapReduce non inventa algoritmi nuovi: inventa un modo di eseguire due operazioni antichissime e semplicissime, map e reduce, in modo sicuro su migliaia di macchine inaffidabili. La domanda del capitolo è quale sia l'astrazione minima che rende possibile tutto questo.

2.1 L'astrazione MapReduce

Prima di MapReduce ogni job di elaborazione su larga scala doveva risolvere da capo gli stessi problemi difficili: come dividere il lavoro tra le macchine, come gestire una macchina che va in crash a metà job, come raccogliere e combinare i risultati parziali. L'intuizione di MapReduce è che questi problemi sono gli stessi per quasi ogni job batch: conviene risolverli una volta per tutte in un framework riusabile, e lasciare al programmatore solo la logica specifica del suo problema.

2.1.1 Radici funzionali

MapReduce prende due primitive dalla programmazione funzionale:

- `map(f, list)` applica la funzione `f` a ogni elemento di una lista, in modo indipendente, e produce una nuova lista;
- `reduce(g, list)` combina gli elementi di una lista a due a due con la funzione `g`, e produce un unico risultato aggregato.

¹J. Dean, S. Ghemawat, *MapReduce: Simplified Data Processing on Large Clusters*, OSDI 2004; poi in *Communications of the ACM* 51(1), 2008.

Il punto cruciale è che map lavora su elementi indipendenti, senza alcun ordine o dipendenza tra loro: è esattamente questo che la rende parallelizzabile su più macchine.

2.1.2 Il modello di programmazione

Definizione (job MapReduce). Dal punto di vista del programmatore, scrivere un **job MapReduce** significa implementare soltanto due funzioni:

```
map(key, value)
  → list(intermediate_key, intermediate_value)
reduce(intermediate_key, list(intermediate_value))
  → list(output_value)
```

Il programmatore non scrive mai codice di scheduling, di rete o di gestione dei guasti: è il framework, non l'utente, a distribuire le chiamate di map e di reduce sul cluster.

La funzione **map** elabora un record di input alla volta, in modo indipendente:

- riceve una singola coppia (chiave, valore) dell'input;
- emette zero, una o molte coppie (chiave, valore) **intermedie**;
- non vede gli altri record dell'input, e non può dipendere dallo stato di altre chiamate di map.

Questa assenza di stato permette al framework di eseguire milioni di chiamate di map in parallelo, in qualunque ordine e su qualunque macchina.

La funzione **reduce** elabora tutti i valori associati a una chiave intermedia, dopo che sono stati raggruppati:

- riceve una chiave intermedia e la lista completa dei valori emessi per quella chiave da tutte le chiamate di map;
- emette il valore o i valori finali per quella chiave.

Chiavi diverse si elaborano in modo indipendente, quindi anche le chiamate di reduce possono girare in parallelo.

2.1.3 Due esempi

Esempio (word count). L'esempio canonico: contare quante volte compare ogni parola in un'enorme collezione di documenti, per esempio salvati come blocchi HDFS. L'output desiderato è, per ogni parola distinta, il numero totale di occorrenze in tutti i documenti.

```
map(document_id, document_text):
    for each word w in document_text:
        emit(w, 1)

reduce(word, list_of_counts):
    total = sum(list_of_counts)
    emit(word, total)
```

Ogni chiamata di map elabora un documento e, per ogni parola che incontra, emette la coppia $(w, 1)$, come a dire «ho visto questa parola una volta». Map non conta ancora nulla: si limita a osservare ed emettere. Ogni chiamata di reduce riceve una parola con la lista di tutti gli 1 emessi per lei in tutti i documenti, e li somma. È il framework a garantire che tutti gli 1 di una parola arrivino alla stessa chiamata di reduce.

Il problema è banale, ma mette in gioco tutti i meccanismi di MapReduce, e nient'altro. L'astrazione però non serve solo a contare.

Esempio (indice invertito). L'indice invertito di un motore di ricerca associa a ogni parola la lista dei documenti che la contengono.

```
map(document_id, document_text):
    for each word w in document_text:
        emit(w, document_id)

reduce(word, list_of_document_ids):
    emit(word, unique_sorted(list_of_document_ids))
```

La struttura è la stessa del word count: cambiano solo i valori emessi da map e la logica di aggregazione di reduce. È qui la vera forza dell'astrazione.

In nessuno dei due esempi il programmatore ha dovuto dire quale macchina esegue quale chiamata, come i dati intermedi si spostano fisicamente da una macchina all'altra, o che cosa succede se una macchina va in crash a metà calcolo. Tutto questo è compito del runtime di MapReduce, descritto nel resto del capitolo.

2.1.4 Determinismo

Le garanzie di tolleranza ai guasti di MapReduce (Sezione 2.4) poggiano su un'ipotesi critica.

Definizione (funzione pura). Le funzioni map e reduce devono essere **pure**: a parità di input producono sempre lo stesso output, e non hanno **effetti collaterali** (*side effects*), cioè non scrivono su stati esterni e non dipendono da stati esterni modificabili.

Se un task fallisce e viene semplicemente rieseguito altrove, il determinismo garantisce che la nuova esecuzione produca risultati identici: è questo che rende «basta riprovare» una strategia valida di tolleranza ai guasti.

2.2 Architettura di esecuzione

Trasformare le funzioni map e reduce dell'utente in un job che gira su un cluster richiede alcuni passi coordinati (Figura 2.1):

1. dividere i dati di input in pezzi indipendenti (*input split*);
2. eseguire un map task per ogni pezzo, distribuendoli sulle macchine disponibili;
3. partizionare l'output intermedio e smistarlo verso i reduce task giusti;
4. eseguire i reduce task, ciascuno responsabile di un sottoinsieme delle chiavi intermedie;
5. raccogliere l'output finale.

Un **map task** elabora un pezzo di input ed emette coppie intermedie; un **reduce task** elabora tutti i valori di una partizione delle chiavi intermedie.

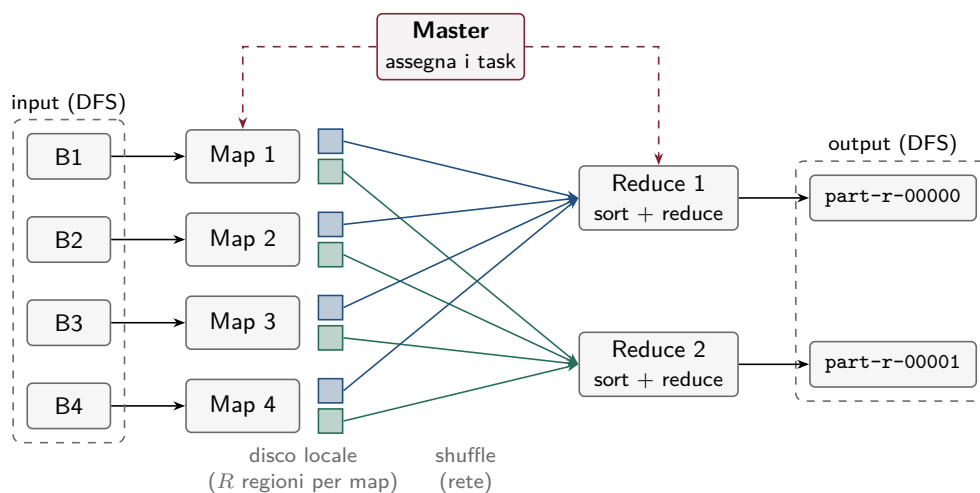


Figura 2.1. Esecuzione di un job con $M = 4$ map task e $R = 2$ reduce task. Ogni map task legge un blocco e scrive il proprio output sul disco locale, già diviso in R regioni (in blu quella per il reduce 1, in verde quella per il reduce 2). Nello shuffle ogni reduce task preleva la propria regione da tutti i map task; poi ordina, applica reduce e scrive un file di output nel DFS.

2.2.1 Il master

Un coordinatore centrale, chiamato **master** o *JobTracker*, gestisce l'esecuzione del job:

- divide l'input in pezzi adatti ai map task;
- assegna map task e reduce task alle macchine libere, i **worker**;
- segue l'avanzamento dei task e li riassegna in caso di guasto (Sezione 2.4).

Concettualmente ha un ruolo analogo al master di GFS o al NameNode di HDFS, ma per il calcolo invece che per lo storage.

2.2.2 Map task e località dei dati

L'input di MapReduce arriva di solito direttamente dal file system distribuito del Capitolo 1, dove è già diviso in blocchi (HDFS) o in chunk (GFS). Per default si crea un map task per ogni blocco di input: l'unità naturale di lavoro parallelo è ereditata dallo strato di storage. Non è una coincidenza: MapReduce è stato progettato insieme a GFS.

Lo scheduling applica il principio di località dei dati (Sezione 1.5): il master esegue ogni map task sulla macchina che ospita già il blocco corrispondente, o vicino a essa. Così si riduce al minimo il traffico di rete della fase map, che legge di gran lunga il volume più grande dell'intero job. Se nessuna macchina perfettamente locale è disponibile, il master ripiega su una macchina dello stesso rack, e solo in ultima istanza su una macchina qualsiasi.

Il numero di map task è di solito molto più grande del numero di macchine: un task per blocco, e i job grandi hanno migliaia di blocchi. Un cluster di, diciamo, 100 macchine esegue queste migliaia di task in molte **ondate** (*waves*) successive. Questa sovra-decomposizione (*over-decomposition*), con tanti task piccoli per macchina, aiuta il bilanciamento del carico e rende più efficace la mitigazione degli straggler (Sezione 2.4.4).

Output intermedio. L'output di un map task non va direttamente a destinazione: viene scritto sul **disco locale** della macchina che ha eseguito il task, non nel file system distribuito. Questi dati intermedi servono solo temporaneamente, alla fase reduce, e scriverli in locale evita un inutile costo di replicazione.

2.2.3 Partizionamento

Prima che i reduce task possano partire, le coppie intermedie vanno raggruppate per chiave e instradate verso il reduce task giusto. Se ne occupa una **funzione di partizionamento**, che associa ogni chiave intermedia a uno degli R reduce task: tutte le coppie con la stessa chiave devono arrivare allo stesso reduce task, qualunque sia il map task che le ha prodotte.

La strategia più comune usa una funzione hash:

```
partition(key) = hash(key) mod num_reduce_tasks
```

Con una funzione hash ragionevolmente uniforme, le chiavi si distribuiscono in modo più o meno equo tra i reduce task; e la funzione è deterministica, quindi la stessa chiave finisce sempre nello stesso reduce task, da qualunque map task provenga. Per esigenze particolari di bilanciamento del carico si possono fornire funzioni di partizionamento personalizzate.

2.2.4 Numero di reduce task

A differenza del numero di map task, che dipende dal numero di blocchi di input, il numero R di reduce task lo sceglie di solito l'utente. Più reduce task danno un parallelismo più fine, ma più file di output e più costo nello shuffle; meno reduce task danno un output più semplice, ma meno parallelismo e task più carichi (Tabella 2.1).

Una regola pratica comune è scegliere un numero di reduce task dell'ordine del numero di worker disponibili, a volte un suo piccolo multiplo.

Tabella 2.1. Il compromesso sul numero di reduce task.

Reduce task	Conseguenza
Troppo pochi	ognuno gestisce una quota enorme delle chiavi: poco parallelismo, e il rischio di un singolo reducer sovraccarico («caldo»)
Troppi	più file di output da gestire, più costo di coordinamento nello shuffle, benefici decrescenti oltre il numero di macchine disponibili

2.2.5 Combiner

Definizione (combiner). Un **combiner** è un'ottimizzazione facoltativa: un piccolo reduce eseguito in locale sull'output di ciascun map task, prima dello shuffle. Applica la stessa logica di aggregazione della funzione reduce, spesso proprio la stessa funzione, quando questa è associativa e commutativa.

Il combiner riduce il volume dei dati intermedi che devono attraversare la rete durante lo shuffle. Nel word count, per esempio, somma localmente le coppie (**w**,1) prodotte dallo stesso map task: se una parola compare mille volte nel blocco, in rete viaggia una sola coppia (**w**,1000) invece di mille coppie (**w**,1).

2.2.6 Output in shard

Come l'input arriva in tanti blocchi, anche l'output viene scritto in tanti file, detti **shard**, e non in un unico file monolitico: ogni reduce task scrive il proprio, con nomi convenzionali come **part-r-00000**, **part-r-00001** e così via. È lo stesso modello dello storage distribuito: un grande dataset si rappresenta naturalmente come un insieme di file. I job successivi possono leggere direttamente questi shard come proprio input distribuito.

2.3 Shuffle, sort e reduce

2.3.1 Lo shuffle

Definizione (shuffle). Lo **shuffle** (letteralmente «rimescolamento») è il trasferimento in rete dell'output intermedio dei map task verso i reduce task giusti. Ogni reduce task deve ricevere tutte le coppie intermedie delle chiavi che gli spettano, qualunque sia la macchina che le ha prodotte.

In generale i dati possono dover viaggiare da ogni map task a ogni reduce task: è uno schema di comunicazione **all-to-all** (da tutti a tutti). La preparazione però comincia prima di qualunque trasferimento. Mentre emette le coppie intermedie,

ogni map task applica subito la funzione di partizionamento a ciascuna chiave, e divide il proprio output sul disco locale in R **regioni**, una per reduce task, già ordinate o raggruppate per destinazione. In seguito ogni reduce task preleva (*pull*) da ciascun map task solo la regione che gli spetta (Figura 2.1).

2.3.2 Perché lo shuffle costa

Lo shuffle è considerato la parte più costosa di un job MapReduce, per più ragioni:

- con M map task e R reduce task, può richiedere fino a $M \times R$ trasferimenti distinti;
- a differenza della fase map, non può sfruttare la località: in generale i dati devono attraversare la rete, perché non c'è modo di rendere ogni reduce task «locale» a tutti i map task contemporaneamente;
- se D è la dimensione complessiva dei dati intermedi, il traffico totale in rete è dell'ordine di D : il parallelismo non lo riduce, lo distribuisce soltanto su tanti trasferimenti più piccoli;
- anche il disco pesa: i dati intermedi vengono scritti sul disco locale dai map task, riletti per essere trasferiti, e spesso riscritti e ordinati dal lato dei reduce.

In molti job reali è lo shuffle, non il calcolo di map o di reduce, a dominare il tempo totale di esecuzione.

Due tecniche pratiche ne riducono l'impatto:

- i **combiner**, che riducono il volume dei dati intermedi prima ancora che debbano essere trasferiti;
- la **compressione** dei dati intermedi prima del trasferimento, che scambia tempo di CPU con meno I/O di rete e di disco: su un cluster lo scambio di solito conviene.

2.3.3 Ordinamento e fase reduce

Dopo aver ricevuto le proprie partizioni da tutti i map task, ogni reduce task ordina i dati per chiave intermedia, così tutti i valori della stessa chiave finiscono uno accanto all'altro. Di solito il framework fa questo ordinamento mentre fonde i dati arrivati dai diversi map task, con un procedimento simile al merge sort.

L'ordinamento è una scelta deliberata per semplificare il lavoro del programmatore: la funzione reduce può contare sul fatto che tutti i valori di una chiave arrivano insieme, in un unico gruppo contiguo, e non deve implementare da sé alcuna logica di raggruppamento. È la stessa filosofia che attraversa tutto il modello: il lavoro difficile di sistema distribuito si fa una volta sola, nel framework.

Con i dati ordinati e raggruppati per chiave, il reduce task elabora un gruppo alla volta:

```
for each (key, sorted_list_of_values) in sorted_input:
    call reduce(key, sorted_list_of_values)
    emit result to output file
```

Ogni chiamata della funzione reduce dell'utente gestisce esattamente una chiave, con la lista completa dei suoi valori. Se una chiave ha un numero enorme di valori, cioè se c'è **sbilanciamento dei dati** (*data skew*), l'uso di memoria può diventare un problema.

Finito il lavoro, il reduce task scrive il proprio output nel file system distribuito, a differenza dell'output dei map task, che restava sul disco locale. Ogni reduce task produce il proprio shard, che viene replicato come qualunque altro file del DFS.

Esempio (word count dall'inizio alla fine). Si segue il word count su due documenti minuscoli, Doc A «the cat sat» e Doc B «the dog ran», con 2 reduce task e partizionamento hash (Figura 2.2).

- **Map.** Il map task di Doc A emette `(the,1)`, `(cat,1)`, `(sat,1)`; quello di Doc B emette `(the,1)`, `(dog,1)`, `(ran,1)`.
- **Partizionamento.** Supponiamo che l'hash mandi «the» e «cat» al reduce 1, e «sat», «dog» e «ran» al reduce 2: sul disco locale di ogni map task le coppie finiscono nella regione del reduce corrispondente.
- **Shuffle e sort.** Il reduce 1 riceve, già ordinate, `(cat,[1])` e `(the,[1,1])`; il reduce 2 riceve `(dog,[1])`, `(ran,[1])` e `(sat,[1])`.
- **Reduce.** Il reduce 1 emette `(cat,1)` e `(the,2)`; il reduce 2 emette `(dog,1)`, `(ran,1)` e `(sat,1)`.

«The» compare in entrambi i documenti, ma le due copie arrivano allo stesso reduce task: è esattamente ciò che la funzione di partizionamento garantisce.

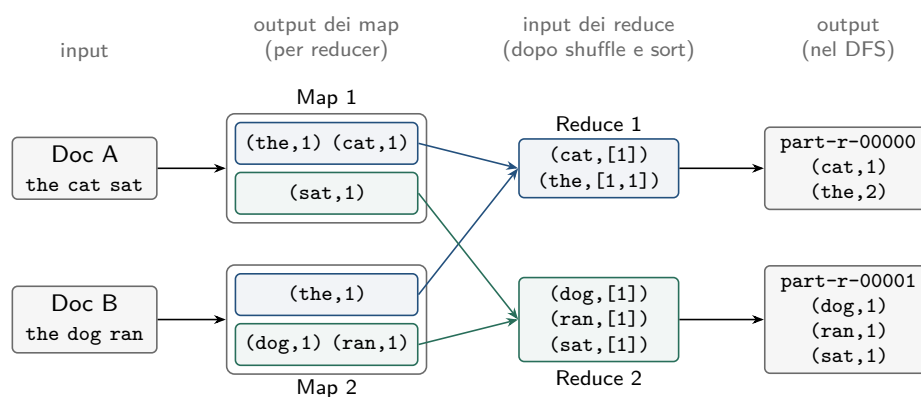


Figura 2.2. Word count su due documenti con 2 reduce task: in blu le chiavi del reduce 1, in verde quelle del reduce 2. Le due coppie `(the,1)`, prodotte da map task diversi, arrivano allo stesso reduce task.

2.4 Tolleranza ai guasti e straggler

Il Capitolo 1 trattava i guasti dello storage: dischi, chunkserver, master. MapReduce aggiunge una seconda dimensione, i guasti durante il calcolo:

- un worker può andare in crash mentre esegue un map task o un reduce task;
- un task può bloccarsi o procedere in modo anomalo, lentissimo, senza fallire davvero: è il problema degli straggler (Sezione 2.4.4);
- il master che coordina il job può a sua volta guastarsi.

2.4.1 Guasto di un worker

Il master rileva i guasti dei worker con lo stesso meccanismo dello strato di storage: i worker gli mandano heartbeat periodici con l'avanzamento dei propri task, e se un worker manca gli heartbeat oltre un certo timeout il master lo considera guasto. A quel punto vanno gestiti i task che il worker stava eseguendo, e in certi casi anche quelli che aveva già completato (Tabella 2.2).

Map task. Il master riassegna semplicemente il map task a un altro worker sano. Lo fa anche se il task originale era già terminato, in parte o del tutto: il suo output stava sul disco locale della macchina guasta, che ora è irraggiungibile. Poiché la funzione map è deterministica, la nuova esecuzione produce un output identico.

Reduce task. Il recupero è simile, ma basta ripercorrere lo shuffle, non il calcolo dei map: il master riassegna il reduce task a un altro worker, che preleva di nuovo i dati intermedi che gli spettano dagli output dei map task, ancora disponibili. Gli output dei map non vanno ricalcolati, solo rilette. Un reduce task già completato, invece, non va rieseguito: il suo output sta già nel file system distribuito.

Tabella 2.2. Che cosa si riesegue quando un worker si guasta.

Task	In corso	Completato
Map	rieseguito su un altro worker	rieseguito: l'output stava sul disco locale del worker guasto
Reduce	rieseguito, rileggendo gli output dei map	non rieseguito: l'output è già nel DFS

2.4.2 Perché la riesecuzione è sicura

Riprovare come strategia di tolleranza ai guasti si regge interamente sul determinismo (Sezione 2.1). Se map e reduce non fossero deterministiche, una riesecuzione potrebbe produrre in silenzio risultati diversi da quelli dell'esecuzione originale, ormai persa. Poiché sono funzioni pure del proprio input, rieseguirle dopo un guasto è indistinguibile, dal punto di vista funzionale, da un task riuscito al primo tentativo. Per questo, nel modello MapReduce, il requisito di determinismo non è negoziabile.

Resta una sottigliezza. Un task può essere rieseguito dopo un guasto, o lanciato in copia in modo speculativo (Sezione 2.4.4), mentre un tentativo precedente è forse ancora in corso. Il framework deve assicurare che, per ogni task, l'output di un solo tentativo sia considerato definitivo, anche se più tentativi girano in contemporanea.

Lo fa con un **commit atomico**: ogni tentativo scrive in una posizione temporanea, e solo un'operazione atomica di rinomina, o di commit, coordinata dal master rende ufficiale l'output di uno dei tentativi.

2.4.3 Guasto del master

Anche il master che coordina il job può guastarsi. Nel progetto originale di MapReduce il guasto del master è raro, perché si tratta di un unico processo che segue il job, e lo si gestisce riavviando l'intero job da capo. È un compromesso deliberato a favore della semplicità: il nodo dei metadati di GFS o HDFS deve sopravvivere per tutta la vita del cluster, mentre il master di MapReduce deve sopravvivere solo per la durata di un job.

2.4.4 Straggler ed esecuzione speculativa

Definizione (straggler). Uno **straggler** (letteralmente «ritardatario») è un task ancora in esecuzione, non andato in crash, ma molto più lento di altri task equivalenti.

Uno straggler non fa scattare il rilevamento dei guasti basato sugli heartbeat, perché il worker è vivo e risponde. Eppure, senza contromisure, il tempo di completamento dell'intero job è vincolato dal task più lento, anche se 999 task su 1000 hanno finito in fretta. Le cause di un task sano ma lento sono diverse:

- **degrado dell'hardware:** un disco o una CPU che rende meno di quanto dovrebbe, senza guastarsi del tutto;
- **contesa delle risorse:** altri job o processi che competono per CPU, disco o rete sulla stessa macchina;
- **sbilanciamento dei dati:** un reduce task a cui tocca un numero insolitamente grande di valori per una chiave, e che impiega quindi un tempo sproporzionato.

La soluzione di MapReduce è l'**esecuzione speculativa** (*speculative execution*):

1. verso la fine del job, il master individua i task molto più lenti della media dei task simili;
2. per ciascuno lancia una **copia di backup** su un altro worker, libero;
3. la prima copia che termina, originale o backup, fornisce il risultato; l'altra viene scartata.

Esempio (l'effetto su un job). Un job ha 1000 map task che impiegano circa 10 secondi ciascuno, ma uno di essi, per un hardware degradato, ne impiega 200 (Figura 2.3). Senza speculazione il job finisce dopo circa 200 secondi, vincolato dallo straggler. Con la speculazione, quando gli altri task hanno finito il master lancia una copia di backup, che termina in circa 10 secondi: il job finisce in 10–20 secondi, vincolato dalla copia di backup. Un solo straggler può allungare il job di un ordine di grandezza, ed è proprio questo il caso che l'esecuzione speculativa colpisce.

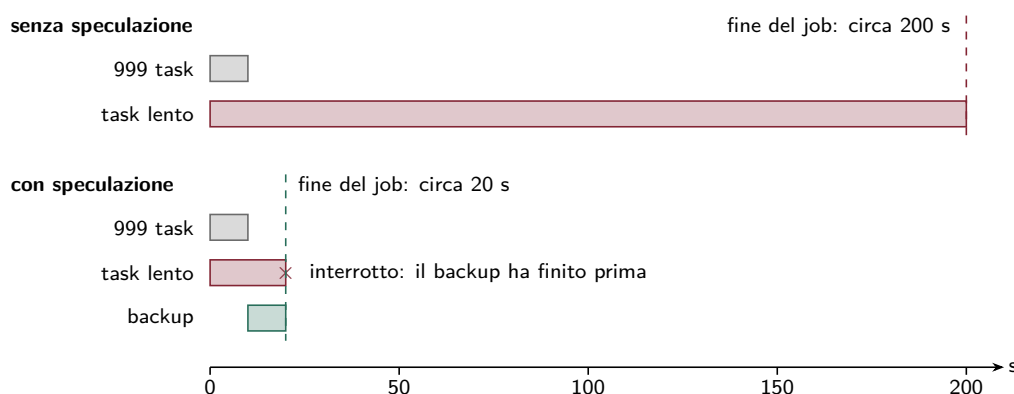


Figura 2.3. L'esempio dell'esecuzione speculativa su un asse dei tempi. Senza speculazione il job aspetta i 200 s del task lento; con la speculazione la copia di backup parte quando gli altri task hanno finito, termina prima dell'originale e il job si chiude in circa 20 s.

L'esecuzione speculativa però non è gratuita:

- **beneficio:** può ridurre drasticamente il tempo di completamento quando pochi task sono anomalmente lenti;
- **costo:** spreca risorse del cluster per copie che alla fine vengono scartate;
- **regolazione:** di solito scatta solo per task molto più lenti dei loro pari, e solo quando nel cluster c'è capacità libera.

2.5 Punti di forza e limiti

La Figura 2.1 riassume la pipeline costruita, meccanismo per meccanismo, in questo capitolo: blocchi di input nel DFS; map task, uno per blocco, eseguiti vicino ai dati; partizionamento locale per hash; shuffle in rete; ordinamento e raggruppamento per chiave; reduce task, uno per partizione di chiavi; output di nuovo nel DFS. Fanno da rete di sicurezza gli heartbeat e la riesecuzione, in qualunque fase, e l'esecuzione speculativa per gli stragglers. La Tabella 2.3 mette a confronto questo strato di calcolo con lo strato di storage del capitolo precedente.

Tabella 2.3. Strato di storage e strato di calcolo a confronto.

Aspetto	Storage (GFS, HDFS)	Calcolo (MapReduce)
Coordinatore	master / NameNode	master / JobTracker
Unità di lavoro	chunkserver / DataNode	map task / reduce task
Rilevamento dei guasti	heartbeat	heartbeat
Recupero	ri-replicazione	riesecuzione
Concetto nuovo	—	esecuzione speculativa (straggler e guasti sono problemi diversi)

Il modello ha avuto successo, alla scala a cui è stato usato, per quattro ragioni:

- **semplicità**: il programmatore scrive due funzioni dall'aspetto sequenziale, non codice di sistema distribuito;
- **tolleranza ai guasti integrata**: determinismo e riesecuzione gestiscono i guasti in modo trasparente;
- **scalabilità grazie alla località**, ereditata direttamente dallo storage GFS e HDFS;
- **generalità**: esprime una gamma sorprendentemente ampia di analisi batch.

Le stesse scelte di progetto portano però limiti concreti:

- **uso intensivo del disco**: i dati intermedi vengono scritti su disco tra ogni map e ogni reduce, un costo pesante per gli algoritmi iterativi, come molti cicli di addestramento del machine learning;
- **struttura rigida a due stadi**: le pipeline complesse richiedono di concatenare più job MapReduce, e ciascuno paga di nuovo per intero il costo dello shuffle;
- **latenza alta**: il modello privilegia il throughput batch, non le interrogazioni interattive o a bassa latenza.

Sono esattamente questi limiti a motivare il modello di esecuzione di Apache Spark, a DAG e in memoria, oggetto della prossima lezione.

Dieci job in catena. Un data scientist deve eseguire 10 job MapReduce in successione, in cui l'output di ciascuno è l'input del successivo, come negli algoritmi iterativi sui grafi. Il costo dominante non è il calcolo: a ogni passo l'output viene scritto nel DFS, e quindi replicato, per essere subito riletto dal job successivo; e ogni job rifà da capo lo shuffle, con i dati intermedi scritti e riletti dal disco. Un modello che tenga i dati in memoria tra un passo e l'altro eviterebbe gran parte di questo I/O: è l'idea da cui parte Spark.

In sintesi

- MapReduce: il programmatore scrive due funzioni, map e reduce; il framework si occupa di parallelismo, scheduling e guasti.
- Map trasforma ogni record in zero o più coppie intermedie, senza stato; reduce aggrega tutti i valori di una chiave. Entrambe devono essere pure: deterministiche e senza effetti collaterali.
- Un map task per blocco di input, eseguito dove sta il blocco (poi nello stesso rack, poi ovunque); l'output intermedio va sul disco locale, diviso in R regioni con $\text{hash}(\text{key}) \bmod R$.
- Lo shuffle è all-to-all (fino a $M \times R$ trasferimenti, volume dell'ordine di D) e pesa su rete e disco: spesso domina il tempo del job. Lo alleggeriscono combiner e compressione.
- Ogni reduce task ordina per chiave il proprio input, chiama reduce una volta per chiave e scrive uno shard nel DFS (`part-r-0000x`).

- Guasti: heartbeat e riesecuzione. I map task si rieseguocono anche se completati, perché l'output locale è perso; i reduce task rileggono soltanto gli output dei map; il commit atomico rende ufficiale un solo tentativo; se cade il master si riavvia il job.
- Uno straggler è lento, non guasto: l'esecuzione speculativa ne lancia una copia di backup verso la fine del job, e vince la prima che termina.
- Limiti: tanto I/O su disco, solo due stadi, latenza alta. Da qui nasce Spark.

Capitolo 3

Apache Spark e i grafi di dataflow

MapReduce ha risolto il problema della sopravvivenza: un calcolo può girare su migliaia di macchine inaffidabili e arrivare in fondo (Capitolo 2). Spark si chiede se, una volta capito come sopravvivere ai guasti, si possa anche essere veloci: tenendo i dati in memoria e ricordando come ricostruirli, invece di replicarli. Il suo modello è descritto nel paper sui Resilient Distributed Dataset.¹ Il capitolo generalizza la pipeline a due stadi di MapReduce in qualcosa di molto più flessibile e centrato sulla memoria: l'astrazione degli RDD, le trasformazioni e le azioni, la valutazione pigra e il DAG di esecuzione, le dipendenze strette e ampie, e infine la tolleranza ai guasti basata sul lineage.

3.1 Oltre la pipeline a due stadi

La Sezione 2.5 si chiudeva con tre limiti di MapReduce: l'uso intensivo del disco, la struttura rigida a due stadi e la latenza alta. Pesano soprattutto sui carichi iterativi, come gli algoritmi sui grafi e i cicli di addestramento del machine learning, che pagano il costo del disco e del rilancio del job a ogni iterazione. I tre limiti hanno una causa comune: MapReduce è progettato attorno a dati residenti su disco e a una pipeline di forma fissa. Spark nasce per superarli.

3.1.1 Il costo dei job concatenati

Le analisi reali richiedono spesso più di un passo map-reduce, e con MapReduce si concatenano più job: il primo legge l'input e scrive il proprio output in HDFS, il secondo lo rilegge e scrive il suo, e così via. Ogni job viene lanciato, schedulato e chiuso per conto suo, e ogni risultato intermedio viene scritto nel file system distribuito per essere poi riletto. Così ogni job in più paga per intero il costo di un job isolato:

- una scrittura completa in HDFS o GFS, con la replicazione, alla fine del job;

¹M. Zaharia et al., *Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing*, NSDI 2012.

- una lettura completa all'inizio del job successivo;
- un nuovo shuffle, anche se le stesse chiavi erano già state raggruppate in un passo precedente.

Una pipeline iterativa di 10 passi fa quindi 10 giri completi su disco e 10 shuffle, anche se solo il primo input e l'ultimo output hanno davvero bisogno di uno storage durevole. È la risposta alla domanda dei dieci job in catena della Sezione 2.5.

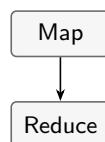
3.1.2 Grafi di calcolo arbitrari

Le pipeline reali hanno bisogno di catene arbitrarie di operazioni, non di un'unica forma $\text{map} \rightarrow \text{reduce}$: filtrare, poi trasformare, poi unire con un altro dataset, poi aggregare, poi filtrare di nuovo. La forma del calcolo è un grafo generale di operazioni, non uno schema fisso a due stadi.

Definizione (DAG). Un **grafo aciclico diretto** (*Directed Acyclic Graph*, DAG) rappresenta un calcolo in modo naturale: i nodi sono le operazioni, o i dataset che producono; gli archi orientati sono le dipendenze tra i dati («l'output di questa operazione è l'input di quella»); l'assenza di cicli garantisce che il calcolo vada sempre avanti, senza mai tornare su se stesso.

In quest'ottica un job MapReduce è semplicemente un DAG con esattamente due stadi. Spark generalizza a DAG di forma e profondità qualsiasi (Figura 3.1).

MapReduce: forma fissa



Spark: DAG di forma qualsiasi

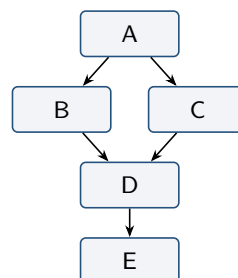


Figura 3.1. Un job MapReduce ha sempre la stessa forma, due stadi in fila; in Spark il calcolo può avere la forma di un DAG qualsiasi, con diramazioni e ricongiunzioni.

3.1.3 Esecuzione in memoria

Il secondo grande cambiamento di Spark è tenere i dati in memoria tra un'operazione e l'altra, quando possibile. I risultati intermedi restano in RAM e per default non vengono scritti su disco: finiscono su uno storage durevole solo quando il programmatore lo chiede esplicitamente, o quando la memoria non basta. È un attacco diretto al primo limite di MapReduce, l'uso intensivo del disco.

Cambia il modello dei costi. L'accesso alla RAM è circa 100 volte più veloce di quello al disco, a parità di dati: rileggere i dati intermedi dalla memoria evita del tutto il giro su disco, e per gli algoritmi iterativi, che scandiscono ripetutamente lo

stesso dataset, la differenza si accumula a ogni iterazione. È l'argomento centrale del paper sugli RDD: tenere i dataset di lavoro residenti in memoria per tutto il calcolo, a più passi o iterativo.

3.1.4 Le motivazioni di Spark

Il paper parte da un'osservazione: i sistemi di dataflow esistenti, come MapReduce, non avevano un modo efficiente di riusare i dati tra più operazioni. La risposta è una nuova astrazione, il Resilient Distributed Dataset (RDD), residente in memoria e insieme tollerante ai guasti, senza affidarsi alla replicazione. Due carichi hanno guidato il progetto:

- gli **algoritmi iterativi**, che scandiscono lo stesso dataset per molte iterazioni: algoritmi sui grafi come PageRank, o cicli di addestramento del machine learning come la discesa del gradiente su un training set fisso. In MapReduce ogni iterazione significherebbe rileggere l'input dal disco, rieseguire un job completo, scrivere l'output e ricominciare;
- le **interrogazioni interattive ed esplorative**: chi esplora un dataset vuole che le query successive rispondano in secondi, non in minuti. Se il dataset viene messo in cache in memoria una volta, le query ripetute evitano del tutto l'I/O su disco; il modello a job batch di MapReduce si presta male a questa interazione rapida.

La Tabella 3.1 riassume la posizione di Spark rispetto a MapReduce.

Tabella 3.1. Spark e MapReduce a confronto.

Aspetto	MapReduce	Spark
Forma della pipeline	fissa a due stadi (map → reduce)	DAG qualsiasi di operazioni
Dati intermedi	scritti su disco tra gli stadi	tenuti in memoria quando possibile
Riuso tra iterazioni	riletti dal disco ogni volta	in cache in memoria tra le iterazioni
Tolleranza ai guasti	riesecuzione dei task falliti	ricalcolo dal lineage (Sezione 3.6)
Adatto a	grandi job batch a passata singola	pipeline iterative, interattive, a più passi

3.2 Resilient Distributed Dataset

Definizione (RDD). Un **Resilient Distributed Dataset** (RDD) è l'astrazione di dati centrale di Spark: una collezione di elementi **partizionata** tra le macchine di un cluster, **immutabile** (una volta creato, il suo contenuto non cambia mai) e **resiliente** (se una partizione va persa, viene recuperata in automatico, senza affidarsi alla replicazione).

Ogni dataset manipolato da un programma Spark, che sia letto dallo storage o derivato da un altro dataset, è un RDD. Concettualmente un RDD si comporta come una versione grande e distribuita di una semplice collezione in memoria, come una lista: gli si applicano operazioni che producono nuovi RDD, ma non se ne possono modificare direttamente gli elementi, e ogni «modifica» produce un RDD nuovo. Questa semplicità, «solo una grande lista distribuita», è voluta: nasconde al programmatore gran parte della complessità distribuita, come facevano le funzioni map e reduce di MapReduce.

3.2.1 Perché l'immutabilità

L'immutabilità non è una restrizione arbitraria: è il fondamento del progetto.

- Poiché un RDD non cambia mai dopo la creazione, il modo in cui è stato creato descrive in modo completo e permanente il suo contenuto.
- Questa storia registrata, «come è stato creato», è esattamente ciò che rende possibile la tolleranza ai guasti basata sul lineage (Sezione 3.6), senza bisogno di replicare i dati.
- L'immutabilità evita anche un'intera classe di bug di concorrenza, quelli dovuti a uno stato mutabile condiviso tra macchine.

Qui sta la differenza di fondo rispetto allo storage del Capitolo 1: la resilienza degli RDD non viene dal replicare i dati su più macchine, come per i blocchi di GFS e HDFS. Spark registra come ogni RDD è stato derivato dai suoi genitori, cioè il suo **lineage**; se una partizione va persa, per esempio per il guasto di una macchina, la ricalcola a partire dal lineage invece di leggerne una replica.

3.2.2 Partizioni

Come i blocchi dello storage e i task di MapReduce, anche gli RDD sono divisi in **partizioni**. Ogni partizione è un sottoinsieme dei dati, che risiede su una macchina alla volta, o è elaborato da una macchina alla volta. Le operazioni su un RDD si applicano in modo indipendente e in parallelo a ciascuna partizione, quindi il numero di partizioni fissa il grado massimo di parallelismo.

Le partizioni sono imparentate con i blocchi (Sezione 1.2), ma sono un'altra cosa. Un RDD creato da un file HDFS ha di solito una partizione per blocco del file; ma lo si può ripartizionare esplicitamente, o creare da sorgenti che non sono file. Il partizionamento è un concetto degli RDD, indipendente dal layout dello storage.

3.2.3 Creare un RDD

Un RDD non è un sistema di storage, ma un'astrazione di calcolo costruita sopra lo storage esistente, indifferente alla provenienza fisica dei dati iniziali. Nasce in due modi:

```
# da uno storage distribuito (per esempio HDFS)
rdd = sc.textFile("hdfs://path/to/data")
# da una collezione in memoria
rdd = sc.parallelize([1, 2, 3, 4, 5])
```

Qui `sc` (lo `SparkContext`) è il punto di accesso a Spark del **driver**, il programma dell'utente, che crea gli RDD e riceve i risultati. Con `textFile` le partizioni derivano dai blocchi del file; con `parallelize` la collezione locale del driver viene divisa in partizioni e distribuita al cluster.

3.2.4 Un'astrazione logica

Conviene distinguere ciò che un RDD rappresenta da ciò che esiste fisicamente in un certo istante. Un RDD è un riferimento logico (*handle*) che descrive «un dataset, partizionato così, derivato da questo calcolo»; i dati veri possono essere materializzati in memoria oppure no, a seconda della valutazione pigra (Sezione 3.4) e delle scelte di caching. Rispetto a un comune array distribuito, un RDD ha tre differenze:

- non ammette aggiornamenti casuali sul posto: non si può modificare l'*i*-esimo elemento, si possono solo derivare nuovi RDD con le trasformazioni;
- il sistema tiene traccia di come è stato prodotto, il lineage, mentre per un array distribuito nessuno lo fa;
- è materializzato in modo pigro: il suo contenuto può non esistere in memoria finché un'azione non ne forza il calcolo.

L'interfaccia di un RDD comprende tre famiglie di operazioni: le **trasformazioni**, che derivano un nuovo RDD da uno esistente; le **azioni**, che fanno partire il calcolo e restituiscono un risultato al driver o scrivono l'output; i **controlli di persistenza**, che chiedono di tenere un RDD in memoria, o su disco, per riusarlo. Rispetto all'input di MapReduce, un insieme fisso di blocchi elaborato da una sola fase map, un RDD si crea dai blocchi allo stesso modo, ma poi si può trasformare, filtrare, unire e aggregare in una catena arbitraria, senza il limite di «una fase map, una fase reduce».

3.2.5 Caching e persistenza

Una funzione centrale per le prestazioni è la possibilità di tenere esplicitamente un RDD in memoria (*cache*) tra un uso e l'altro:

```
rdd = sc.textFile("hdfs://path/to/data")
rdd.cache()    # da tenere in memoria
```

Senza caching Spark potrebbe dover ricalcolare l'RDD da zero, tramite il lineage, ogni volta che viene riusato; con il caching le partizioni calcolate restano in memoria, pronte per le operazioni successive. È ciò che rende possibili i carichi iterativi e

interattivi della Sezione 3.1. Spark offre diversi livelli di persistenza, che bilanciano velocità e capacità (Tabella 3.2); la scelta dipende da quanto è grande il dataset rispetto alla memoria del cluster.

Tabella 3.2. Livelli di persistenza di un RDD.

Livello	Comportamento
Solo memoria	il più veloce; le partizioni che non ci stanno vengono ricalcolate quando servono
Memoria e disco	le partizioni in eccesso finiscono sul disco locale invece di essere scartate
Solo disco	nessuna pressione sulla memoria, al prezzo di un accesso alla velocità del disco

3.3 Trasformazioni e azioni

Ogni operazione che un programma Spark esegue su un RDD appartiene a una e una sola di due categorie.

Definizione (trasformazione e azione). Una **trasformazione** prende uno o più RDD e produce un nuovo RDD, che descrive un dataset derivato. Un'**azione** fa partire il calcolo e produce un risultato fuori dall'astrazione degli RDD: lo restituisce al driver, oppure lo scrive nello storage.

Le trasformazioni sono funzioni deterministiche sui dati, come la funzione `map` del Capitolo 2, e per l'immutabilità non modificano mai l'RDD di partenza:

```
rdd1 = sc.textFile("data.txt")
rdd2 = rdd1.filter(lambda line: "error" in line)
# rdd1 non cambia; rdd2 descrive il risultato filtrato
```

`rdd1` resta esattamente com'era, e `rdd2` è un dataset distinto, appena descritto: una catena di trasformazioni produce semplicemente una catena di descrizioni di RDD.

Le azioni invece fanno calcolare davvero i risultati. Chiamare una trasformazione registra soltanto «ecco un nuovo RDD, derivato così», senza elaborare alcun dato; chiamare un'azione fa eseguire al cluster il calcolo necessario. Questa separazione tra descrivere il calcolo e farlo partire è la base della valutazione pigra (Sezione 3.4), e apre un'occasione di ottimizzazione: Spark vede l'intera catena di trasformazioni prima che ne parta una, e può ottimizzare la pipeline come un tutto, invece di eseguire ogni operazione appena viene chiamata, una alla volta. La Tabella 3.3 raccoglie le trasformazioni e le azioni più comuni.

Esempio (word count in Spark). L'esempio canonico del Capitolo 2, scritto con gli RDD:

```
lines = sc.textFile("hdfs://documents/")
words = lines.flatMap(lambda line: line.split())
pairs = words.map(lambda word: (word, 1))
counts = pairs.reduceByKey(lambda a, b: a + b)
```

`flatMap`, `map` e `reduceByKey` sono tutte trasformazioni, e a questo punto non è stato calcolato nulla: `lines`, `words`, `pairs` e `counts` sono solo descrizioni di RDD, che dicono come calcolare il risultato, non il risultato. Nessun dato è stato letto da HDFS, nessun `flatMap` è stato eseguito, nessun conteggio è stato fatto. Il calcolo parte solo con un'azione:

```
result = counts.collect() # azione: esegue l'intera catena
```

Questa riga fa leggere l'input da HDFS, eseguire `flatMap`, `map` e `reduceByKey`, compreso lo shuffle necessario, e infine raccogliere i risultati nel driver. Tutte le trasformazioni definite prima vengono eseguite insieme, in quest'unico calcolo.

Due reduce diversi. I nomi possono confondere. In Spark `reduce(f)` è un'azione: aggrega tutti gli elementi di un RDD in un unico valore, restituito al driver. `reduceByKey(f)` è invece una trasformazione: aggrega i valori di ogni chiave e produce un nuovo RDD di coppie. Nessuna delle due coincide con la funzione `reduce` di MapReduce, che l'utente scrive e il framework chiama una volta per chiave: nel word count il suo ruolo lo svolge `reduceByKey`.

Tabella 3.3. Trasformazioni e azioni più comuni.

Operazione	Effetto
<i>Trasformazioni</i>	
<code>map(f)</code>	applica <code>f</code> a ogni elemento
<code>filter(f)</code>	tiene gli elementi per cui <code>f</code> è vera
<code>flatMap(f)</code>	applica <code>f</code> e appiattisce le liste risultanti
<code>distinct()</code>	elimina i duplicati
<code>union(other)</code>	unisce all'RDD un altro RDD
<code>groupByKey()</code>	raggruppa i valori per chiave
<code>reduceByKey(f)</code>	aggrega con <code>f</code> i valori di ogni chiave
<code>join(other)</code>	unisce per chiave due RDD di coppie chiave-valore
<i>Azioni</i>	
<code>count()</code>	restituisce il numero di elementi
<code>collect()</code>	restituisce tutti gli elementi al driver
<code>take(n)</code>	restituisce i primi <code>n</code> elementi
<code>reduce(f)</code>	aggrega tutti gli elementi con <code>f</code>
<code>saveAsTextFile(path)</code>	scrive il contenuto dell'RDD nello storage

La pigrizia però non rende le trasformazioni gratuite: ne rimanda il costo. Ogni trasformazione deve comunque essere eseguita, come parte dell'azione che alla fine

fa partire la catena; cambia quando il lavoro avviene, e quanta visibilità ha Spark sull'intera pipeline prima di decidere come eseguirla.

3.4 Valutazione pigra e DAG di esecuzione

Definizione (valutazione pigra). La **valutazione pigra** (*lazy evaluation*) è il principio per cui le trasformazioni vengono registrate come un piano, non eseguite subito: chiamare `map`, `filter` o qualunque altra trasformazione aggiorna soltanto la descrizione interna del calcolo, e nessuna partizione viene letta, trasformata o scritta finché un'azione non lo impone.

È il meccanismo dietro a quanto visto nel word count. Rimandare l'esecuzione è una scelta di progetto con benefici concreti:

- Spark vede l'intera pipeline di trasformazioni prima di scegliere una strategia di esecuzione;
- questa visibilità permette ottimizzazioni tra operazioni diverse, impossibili se ogni trasformazione venisse eseguita subito e in isolamento;
- si evita lavoro sprecato: se un'azione ha bisogno solo di una parte dei dati, come `take(5)`, a volte Spark riesce a non calcolare più del necessario.

3.4.1 Il piano logico e il DAG

Mentre le trasformazioni si concatenano, Spark costruisce un **piano di esecuzione logico**: ogni RDD registra un riferimento ai propri RDD genitori e la trasformazione che lo ha prodotto. Ne risulta una catena, o più in generale un grafo, di RDD che puntano a come sono stati derivati: è esattamente il lineage della Sezione 3.2, visto qui nel suo ruolo di piano di esecuzione.

Questa struttura forma un DAG: ogni RDD è un nodo e ogni trasformazione un arco orientato dagli RDD genitori al nuovo RDD; le operazioni che combinano più RDD, come `join` e `union`, creano nodi con più archi entranti. Nel word count il DAG è una catena, `lines` → `words` → `pairs` → `counts`, e l'intero grafo esiste nella memoria di Spark prima che venga chiamata `collect()`: è il piano logico, non ancora eseguito.

3.4.2 Ottimizzazioni

Vedere tutto il DAG prima dell'esecuzione permette diverse ottimizzazioni:

- la **fusione degli operatori** (*operator fusion*, o *pipelining*): concatenare operazioni compatibili, per non materializzare risultati intermedi inutili;
- **evitare calcoli inutili**: saltare le parti del DAG che non servono al risultato richiesto dall'azione;
- **scegliere la strategia fisica**: decidere come raggruppare le operazioni in stage, in base alla struttura delle dipendenze (Sezione 3.5).

Quando una sequenza di trasformazioni si può applicare allo stesso elemento senza aspettare gli altri elementi o le altre partizioni, Spark la esegue in pipeline. In

```
words.map(f).filter(g).map(h)
```

invece di materializzare l'intero risultato intermedio dopo `map(f)`, e poi di nuovo dopo `filter(g)`, Spark applica `f`, poi `g`, poi `h` a ogni elemento, in una sola passata per partizione, risparmiando memoria e passate sui dati. È un vantaggio diretto sui job concatenati di MapReduce: lì l'output completo di ogni job doveva essere materializzato, cioè scritto in HDFS, prima che partisse il successivo; qui i risultati intermedi di trasformazioni compatibili possono non essere mai materializzati del tutto, perché i dati attraversano più operazioni in una sola passata.

3.4.3 Dal DAG logico all'esecuzione

Quando finalmente viene chiamata un'azione, Spark trasforma il DAG logico in qualcosa di eseguibile sul cluster:

1. il **DAG scheduler** esamina l'intera catena di dipendenze che porta all'azione richiesta;
2. divide il DAG in un insieme di **stage**;
3. divide ogni stage in **task**, uno per partizione, e li distribuisce sul cluster, come lo scheduling dei task di MapReduce (Sezione 2.2).

Definizione (stage). Uno **stage** è un gruppo di operazioni che si possono eseguire insieme, senza bisogno di rimescolare i dati in rete tra l'una e l'altra. Le operazioni di uno stage vengono eseguite in pipeline; un nuovo stage comincia ogni volta che il DAG richiede uno shuffle, cioè una redistribuzione dei dati tra le partizioni prima che l'operazione successiva possa procedere.

I confini tra gli stage non sono arbitrari: cadono esattamente dove un'operazione non si può calcolare con i soli dati locali della partizione. Operazioni come `map` o `filter` si concatenano liberamente, perché non hanno bisogno di dati di altre partizioni; operazioni come `reduceByKey` o `join` devono raggruppare di nuovo i dati tra le partizioni, esattamente come lo shuffle di MapReduce (Sezione 2.3). La Sezione 3.5 formalizza questa distinzione.

Esempio (word count esteso). Si aggiunge un passo al word count: dopo il conteggio si scartano le parole rare.

```
lines = sc.textFile("hdfs://documents/")
words = lines.flatMap(lambda line: line.split())
pairs = words.map(lambda word: (word, 1))
counts = pairs.reduceByKey(lambda a, b: a + b)
frequent = counts.filter(lambda kv: kv[1] > 100)
result = frequent.collect() # azione: esegue l'intera catena
```

Sono cinque passi concatenati, eseguiti come un unico DAG coordinato quando

finalmente si chiama `collect()`. In quanti stage si dividano lo mostra la Sezione 3.5.

La Tabella 3.4 riassume come cambia l'esecuzione rispetto al modello a due stadi di MapReduce.

Tabella 3.4. L'esecuzione in MapReduce e in Spark.

Aspetto	MapReduce	Spark
Quando avviene il lavoro	subito, mentre ogni job gira	rimandato finché non si chiama un'azione
Visibilità sulla pipeline	un job alla volta, isolato	l'intera catena di trasformazioni, prima dell'esecuzione
Pipeline a più passi	job separati, ognuno con il suo giro su disco	un solo DAG, ottimizzato ed eseguito insieme
Profondità della pipeline	esattamente map, poi reduce	un numero arbitrario di operazioni concatenate
Dati intermedi	su disco, tra un job e l'altro	in pipeline in memoria, dove possibile
Dove avviene lo shuffle	sempre, una volta, tra map e reduce	solo ai confini delle dipendenze ampie

3.5 Dipendenze strette e ampie

Una **dipendenza** descrive il rapporto tra le partizioni di un RDD figlio e le partizioni degli RDD genitori da cui dipendono: è la versione precisa e formale della storia delle trasformazioni introdotta come lineage. Il tipo di dipendenza determina quanto costa eseguire una trasformazione e se questa impone un nuovo stage.

Definizione (dipendenza stretta e ampia). Una dipendenza è **stretta** (*narrow*) quando ogni partizione dell'RDD figlio dipende da un numero piccolo e fisso di partizioni dell'RDD genitore, di solito una sola. È **ampia** (*wide*) quando una partizione del figlio può dipendere da dati di più partizioni del genitore, potenzialmente di tutte.

`map`, `filter` e `flatMap` sono strette: ogni partizione di output si calcola solo dalla partizione di input corrispondente, e nessun dato si sposta tra partizioni che stanno su macchine diverse. `groupByKey`, `reduceByKey` e `join` sono ampie: per produrre una sola partizione di output possono servire dati sparsi in tutte le partizioni di input (Figura 3.2). È la stessa struttura del movimento di dati all-to-all dello shuffle di MapReduce (Sezione 2.3).

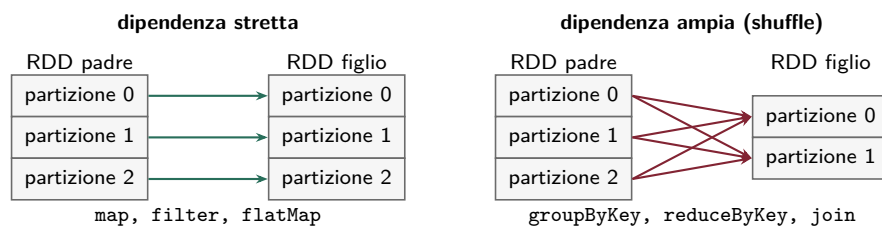


Figura 3.2. Dipendenze strette e ampie. Nella stretta ogni partizione del figlio dipende da una sola partizione del padre, e il calcolo resta locale; nell'ampia ogni partizione del figlio può ricevere dati da tutte quelle del padre, e serve uno shuffle in rete.

3.5.1 Pipeline e shuffle

La distinzione è il meccanismo preciso dietro ai confini degli stage.

- Le dipendenze strette permettono di concatenare le operazioni in pipeline in sicurezza: una catena di trasformazioni strette si calcola tutta dentro una partizione, su una macchina, in una passata. In `data.map(f1).filter(f2).map(f3)` le tre operazioni si applicano a ogni partizione in modo indipendente, senza coordinamento tra partizioni. È il pipelining della Sezione 3.4, e ora si vede perché è possibile: ogni passo è una dipendenza stretta.
- Le dipendenze ampie richiedono uno shuffle: non si può calcolare una trasformazione come `reduceByKey` senza prima raggruppare di nuovo i dati per chiave in tutto il cluster, con partizionamento, trasferimento in rete e raggruppamento, come in MapReduce. Spark non evita il costo dello shuffle: lo confina ai soli punti del DAG in cui è inevitabile, invece di imporlo dopo ogni job come la concatenazione di job MapReduce.

Si può quindi dire con precisione dove finisce uno stage: uno stage è una sequenza massimale di trasformazioni con dipendenze strette, eseguibili in pipeline, e un nuovo stage comincia esattamente a ogni dipendenza ampia, perché per calcolarla serve che l'output dello shuffle sia disponibile.

Esempio (gli stage del word count esteso). Nel DAG del word count esteso (Sezione 3.4) `flatMap`, `map` e `filter` sono strette, e formano segmenti in pipeline dentro gli stage; `reduceByKey` è ampia, ed è lì che cade il confine tra gli stage, con lo shuffle. La catena di cinque passi si riduce quindi a due soli stage fisici (Figura 3.3).

3.5.2 Il costo delle dipendenze ampie

Le dipendenze ampie hanno gli stessi costi dello shuffle di MapReduce:

- consumano banda di rete in proporzione al volume dei dati redistribuiti;
- richiedono I/O su disco per scrivere e rileggere i dati dello shuffle;²

²Le slide dicono che Spark può riversare su disco i dati dello shuffle quando la memoria scarseggia. In realtà l'output del lato map di uno shuffle viene sempre scritto sul disco locale, come l'output dei

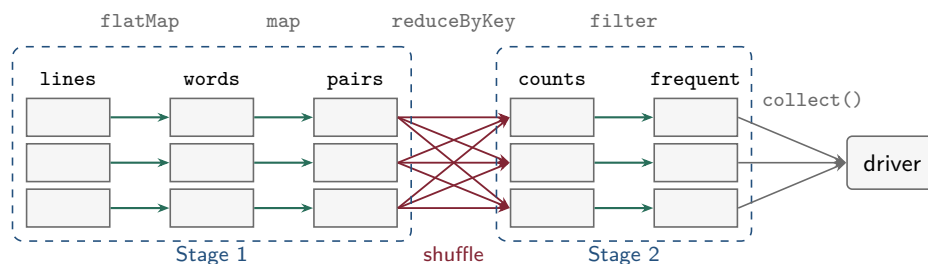


Figura 3.3. Gli stage del word count esteso, con tre partizioni per RDD. Le trasformazioni strette (in verde) stanno in pipeline dentro uno stage; `reduceByKey` (in rosso) è una dipendenza ampia, quindi uno shuffle e l'inizio del secondo stage. L'azione `collect()` riporta i risultati al driver.

- sono un punto di sincronizzazione naturale: tutte le partizioni che contribuiscono a una dipendenza ampia devono essere completate prima che lo stage successivo possa procedere del tutto.

Poiché sono la voce di costo dominante, i programmi Spark si scrivono in modo da ridurne numero e impatto:

- si preferisce `reduceByKey` a `groupByKey` seguito da un'aggregazione separata, perché `reduceByKey` può aggregare parzialmente in locale prima dello shuffle, proprio come un combiner di MapReduce (Sezione 2.2.5);
- dove possibile, si spostano le operazioni strette, come filtri e map, prima di quelle ampie, per ridurre il volume di dati da rimescolare.

3.6 Lineage e tolleranza ai guasti

Come per lo storage e per MapReduce, i guasti su un cluster sono routine: una macchina che ospita partizioni di uno o più RDD può andare in crash a metà calcolo. Le risposte viste finora sono due: lo storage replica i dati su più macchine (Capitolo 1), MapReduce riesegue i task falliti a partire dall'input originale (Capitolo 2). Spark costruisce su entrambe e ricalcola soltanto le partizioni perse, usando il lineage registrato, senza bisogno di copie replicate di ogni RDD intermedio.

Definizione (lineage). Il **lineage** (letteralmente «discendenza») di un RDD è la catena registrata di trasformazioni che descrive esattamente come le sue partizioni sono state derivate dagli RDD genitori: ogni RDD ricorda da quali genitori viene e quale trasformazione, con quale funzione di partizionamento, lo ha prodotto.

È la struttura del DAG costruita nelle Sezioni 3.4 e 3.5, vista ora come meccanismo di tolleranza ai guasti invece che di pianificazione. Qui sta la differenza centrale rispetto a GFS e HDFS: quelli tengono più copie fisiche dei dati, Spark tiene una

map in MapReduce: lo dice anche il paper sugli RDD, che motiva la scelta con la semplicità del recupero dopo un guasto. Sotto pressione di memoria, Spark riversa su disco anche i buffer che usa per ordinare e aggregare.

descrizione compatta di come ricrearli e li ricalcola quando serve. L'informazione di lineage è minuscola rispetto ai dati che descrive: pochi riferimenti a trasformazioni, qualunque sia la dimensione del dataset.

3.6.1 Ricalcolo di una partizione

Se una partizione di un RDD va persa, per esempio perché la macchina che la ospitava si guasta, Spark ricostruisce esattamente quella partizione:

1. individua dal lineage le partizioni genitrici da cui dipende;
2. se queste sono ancora disponibili (in memoria, su disco, o a loro volta ricalcolabili), ricalcola la partizione persa riapplicando la trasformazione;
3. se manca anche una partizione genitrice, risale ancora la catena del lineage, finché non trova dati disponibili.

Esempio (una partizione persa). Si consideri la catena RDD originale $\rightarrow A \rightarrow B \rightarrow C$, dove A si ottiene con `map`, B con `filter` e C con `reduceByKey`, e supponiamo che vada persa una partizione di B, perché la sua macchina si è guastata (Figura 3.4). Spark risale il lineage: la partizione di B dipende da una partizione nota di A. Se questa è ancora disponibile, basta riapplicarle `filter` per ricreare la partizione persa, senza toccare nessun'altra partizione.

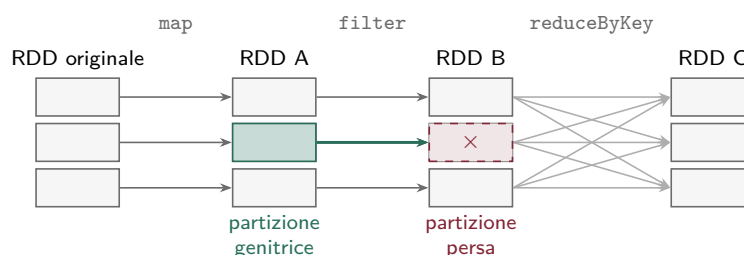


Figura 3.4. Ricalcolo dal lineage. Si perde una partizione di B (in rosso); la si ricostruisce riapplicando `filter` alla sola partizione di A da cui dipende (in verde), senza toccare le altre.

3.6.2 Tre strategie a confronto

La Tabella 3.5 mette a confronto le tre strategie di tolleranza ai guasti viste nel modulo. Rispetto a MapReduce, l'approccio di Spark è una generalizzazione diretta del «basta ricalcolare», applicata partizione per partizione lungo un DAG arbitrariamente lungo. Rispetto alla replicazione è un vero compromesso, non un miglioramento netto: si risparmia spazio, ma si paga il ricalcolo.

Replicare ogni RDD intermedio sarebbe proibitivo, visto quanti ne crea un programma Spark tipico: una pipeline di dieci trasformazioni concatenate richiederebbe di memorizzare, e tenere sincronizzati, dieci dataset replicati. Il lineage invece memorizza per ogni RDD una quantità di metadati piccola e costante, qualunque

Tabella 3.5. Tre strategie di tolleranza ai guasti.

Aspetto	GFS e HDFS	MapReduce	Spark
Meccanismo	replicazione fisica, di solito 3×	riesecuzione dei task falliti	ricalcolo dal lineage
Che cosa si ricalcola	nulla: si legge una replica	il task fallito, per intero	solo le partizioni perse, seguendo il lineage
Spazio in più	3 volte il volume dei dati	—	minimo: il grafo delle trasformazioni
Costo del recupero	basso: leggere una replica esistente	—	ricalcolare dal lineage, lento se la catena è lunga
Pipeline a più passi	—	il guasto di ogni job si gestisce a parte	un'unica catena di lineage copre tutta la pipeline

sia la dimensione dei dati, e costa molto meno nel caso comune delle lunghe catene di trasformazioni.

3.6.3 Catene lunghe e checkpointing

Il recupero dal lineage ha però un limite. Se la catena è molto lunga, ricalcolare una partizione persa verso la fine può richiedere di ricalcolare gran parte di tutto ciò che la precede: il costo cresce con la profondità del DAG, ed è un problema soprattutto per gli algoritmi iterativi, dove lo stesso RDD viene trasformato di nuovo a ogni iterazione.

Per limitarlo Spark permette il **checkpointing** esplicito. Un checkpoint scrive il contenuto attuale di un RDD su uno storage durevole e replicato, come HDFS, nello spirito della replicazione del Capitolo 1. Dopo il checkpoint la catena del lineage di fatto riparte da lì: un ricalcolo futuro risale solo fino al checkpoint, non fino all'input originale. È un compromesso deliberato: si paga subito un costo di storage e di replicazione, in cambio di un costo di ricalcolo limitato in seguito.

3.6.4 Recupero e scheduling

Il recupero dai guasti si integra con lo scheduling delle Sezioni 3.4 e 3.5. Quando un task fallisce, per esempio per il crash di una macchina, il DAG scheduler individua esattamente quali partizioni sono andate perse e rischedula nuovi task per ricalcolare solo quelle, usando la stessa struttura di stage e dipendenze già stabilita per l'esecuzione normale. Il recupero riusa quindi la stessa macchina del DAG: non c'è un sistema di recupero separato, gestito come caso speciale.

3.7 Il modello di esecuzione completo

Mettendo insieme i pezzi, un programma Spark viene eseguito così:

1. le trasformazioni (`map`, `filter`, `join...`) vengono registrate in modo pigro;
2. formano un DAG logico, il lineage degli RDD;
3. quando un'azione lo richiede, il DAG scheduler divide il DAG in stage ai confini delle dipendenze ampie, e ogni stage è una catena di trasformazioni strette in pipeline;
4. ogni stage si divide in task, uno per partizione;
5. i task vengono distribuiti sul cluster, dove possibile vicino ai dati;
6. l'azione fa partire l'esecuzione, e i risultati tornano al driver o vengono scritti nello storage.

Il DAG scheduler. È il componente di coordinamento di Spark, concettualmente analogo al master (o JobTracker) di MapReduce. Riceve il grafo del lineage quando viene chiamata un'azione, determina i confini degli stage in base alle dipendenze strette e ampie, e sottomette i task di uno stage solo quando le sue dipendenze in ingresso sono soddisfatte. Il suo output è un piano concreto ed eseguibile, costruito in quattro passi:

1. percorre all'indietro il grafo del lineage, a partire dall'RDD bersaglio dell'azione;
2. raggruppa le catene di dipendenze strette in stage eseguiti in pipeline;
3. ordina gli stage in modo che gli input che ciascuno riceve da dipendenze ampie, cioè gli output degli shuffle, siano pronti prima che parta;
4. dentro ogni stage crea un task per partizione, pronto per essere assegnato alle macchine del cluster.

Località dei dati. Come GFS, HDFS e MapReduce (Sezione 1.5), anche lo scheduler dei task di Spark preferisce eseguire i task vicino ai dati. Nel primo stage di un RDD, quello che legge da HDFS o GFS, i task vanno sulle macchine che ospitano i blocchi corrispondenti, esattamente come i map task; negli stage successivi Spark preferisce le macchine che hanno già in memoria le partizioni genitrici, magari in cache, per evitare spostamenti di dati inutili.

Punti di forza e limiti. Rispetto a MapReduce, il modello di Spark offre:

- **generalità:** DAG arbitrari di operazioni, non una forma fissa a due stadi;
- **prestazioni per i carichi iterativi e interattivi:** il caching in memoria evita i giri ripetuti su disco;
- **tolleranza ai guasti efficiente:** il ricalcolo dal lineage evita il costo in spazio di replicare ogni dataset intermedio;
- **esecuzione ottimizzabile:** vedere l'intera pipeline prima di eseguirla permette il pipelining ed evita lavoro inutile.

Restano alcuni limiti: l'API degli RDD è di basso livello, e le interrogazioni di tipo relazionale diventano prolisse; gli RDD non conoscono la struttura dei dati, come schemi e colonne; il modello batch non gestisce in modo nativo flussi di dati continui e illimitati. Sistemi più recenti vanno oltre: l'elaborazione continua di stream (Apache Flink), i grafi di task dinamici per l'AI distribuita (Ray), le astrazioni di storage lakehouse (Delta Lake).

In sintesi

- MapReduce è rigido (due stadi), pesa sul disco e ha latenza alta: con i job concatenati ogni passo paga scrittura e lettura dal DFS e un nuovo shuffle. Spark generalizza a DAG arbitrari e tiene i dati intermedi in memoria.
- Un RDD è una collezione distribuita partizionata, immutabile e resiliente; le partizioni sono l'unità di parallelismo, e `cache()` tiene un RDD in memoria per riusarlo.
- Le trasformazioni (`map`, `filter`, `reduceByKey`, `join...`) sono pigre e producono nuovi RDD; le azioni (`count`, `collect`, `take`, `saveAsTextFile...`) fanno partire il calcolo.
- La valutazione pigra costruisce il DAG logico prima dell'esecuzione e permette di ottimizzare, per esempio fondendo più trasformazioni in una sola passata per partizione.
- Nelle dipendenze strette ogni partizione del figlio viene da poche partizioni del genitore: si eseguono in pipeline dentro uno stage. Nelle dipendenze ampie viene da molte o da tutte: servono uno shuffle e un nuovo stage. Il word count esteso ha due stage.
- Lo shuffle costa come in MapReduce; lo si riduce con `reduceByKey` al posto di `groupByKey`, che aggrega in locale come un combiner, e filtrando prima delle operazioni ampie.
- Guasti: si ricalcolano dal lineage solo le partizioni perse, risalendo la catena. Costa poco in spazio, ma il ricalcolo cresce con la lunghezza della catena; il checkpointing la accorcia salvando un RDD su storage replicato.
- Il DAG scheduler, analogo al master di MapReduce, divide il DAG in stage alle dipendenze ampie e crea un task per partizione, assegnato rispettando la località dei dati.